Subject Code	Subject Name	Category	L	T	P	S	Credits	Inst.Hours	Marks		
									CIA	External	Total
23BCE5E1	**Artificial Intelligence**	DSE-IA	4	-	-	-	3	4	25	75	100

Course Objective	
C1	To learn various concepts of AI Techniques.
C2	To learn various Search Algorithm in AI.
C3	To learn probabilistic reasoning and model sin AI.
C4	To learn about Markov Decision Process.
C5	To learn various type of Reinforcement learning.

	Contents	**No.of Hours**
UNIT I	Introduction: Concept of AI, history, current status, scope, agents, environments, Problem Formulations, Review of tree and graph structures, State space representation, Search graph and Search tree	12
UNIT II	Search Algorithms: Random search, Search with closed and open list, Depth first and Breadth first search, Heuristic search, Best first search, A* algorithm, Game Search	12
UNIT III	Probabilistic Reasoning: Probability, conditional probability, Bayes Rule, Bayesian Networks- representation, construction and inference, temporal model, hidden Markov model.	12
UNIT IV	Markov Decision process: MDP formulation, utility theory, utility functions, value iteration, policy iteration and partially observable MDPs.	12
UNIT V	Reinforcement Learning: Passive reinforcement learning, direct utility estimation, adaptive dynamic programming,temporal difference learning, active reinforcement learning- Q learning	12
	Total	**60**

R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.

UNIT I

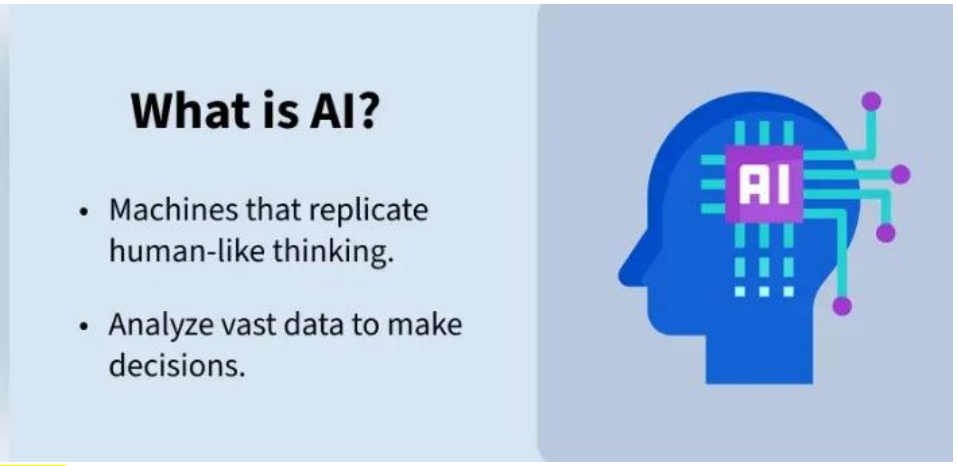
Introduction:

- **Artificial Intelligence (AI)** is a branch of computer science that focuses on creating **machines capable of performing tasks that require human intelligence**, such as:

- Learning from experience
- Reasoning and problem solving
- Understanding language
- Perceiving environment and taking action

In simple words:

AI makes computers “think” and “act” smart like humans.



Concept of AI

AI is based on core concepts and technologies that enable machines to learn, reason and make decisions on their own. Let's see some of the concepts:

1. Machine Learning (ML)

Machine Learning is a subset of artificial intelligence (AI) that focuses on building systems that can learn from and make decisions based on data. Instead of being explicitly programmed to perform a task, a machine learning model uses algorithms to identify patterns within data and improve its performance over time without human intervention.

2. Generative AI

Generative AI is designed to create new content whether it's text, images, music or video. Unlike traditional AI which typically focuses on analyzing and classifying data, it goes a step further by using patterns it has learned from large datasets to generate new original outputs. It "creates" rather than just "recognizes."

3. Natural Language Processing (NLP)

Natural Language Processing (NLP) allows machines to understand and interact with human language in a way that feels natural. It enables speech recognition systems like Siri or Alexa to interpret what we say and respond accordingly. It combines linguistics and computer science to help computers process, understand and generate human language allowing for tasks like language translation, sentiment analysis and real-time conversation.

4. Expert Systems

Expert Systems are designed to simulate the decision-making ability of human experts. These systems use a set of predefined "if-then" rules and knowledge from specialists in specific fields to make informed decisions similar to how a medical professional would diagnose a disease. They are useful in areas where expert knowledge is important but not always easily accessible.

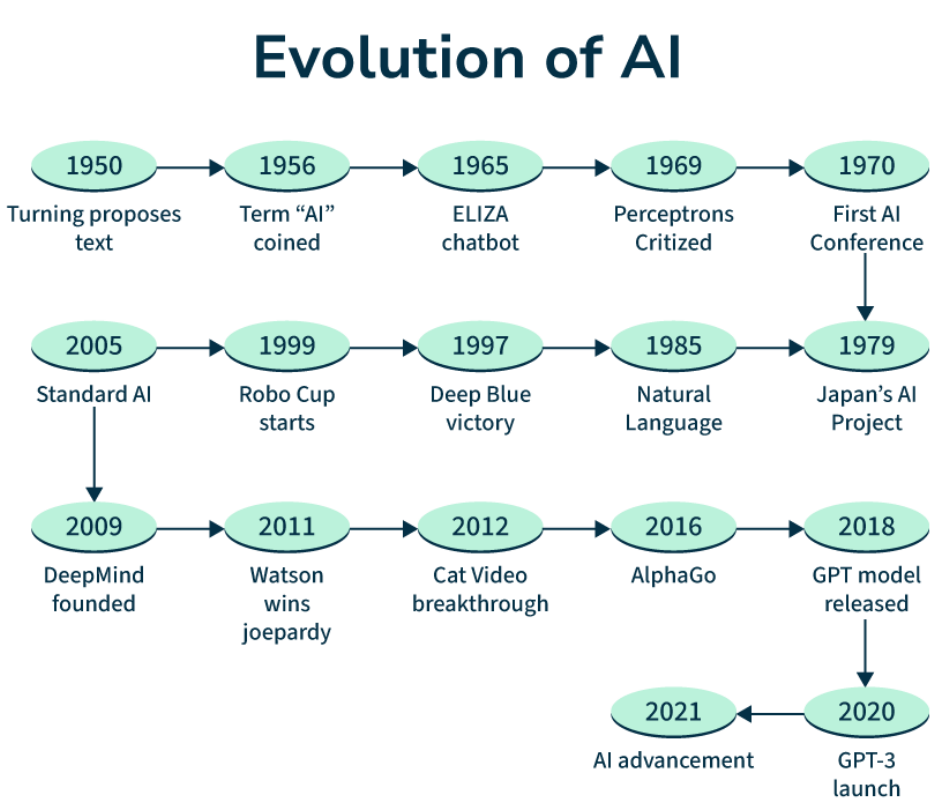
Working of Artificial Intelligence

AI works by simulating intelligent behavior to perform tasks autonomously. The process involves several steps that help machines learn, make decisions and improve over time:

1. **Data Collection**: AI systems rely on large sets of data which could include images, text or sensor readings. For example, teaching an AI to recognize cats, we collect a dataset of labeled cat images.
2. **Processing and Learning**: It uses algorithms to analyze data and identify patterns. For example, it learns to recognize key features like a cat's shape, ears or whiskers helping it understand the data.
3. **Model Training**: The AI model is trained using the data, adjusting its internal settings to improve its predictions. With more data, the model becomes more accurate and better at recognizing new examples like unseen images of cats.
4. **Decision Making**: Once trained, it can use what it has learned to make decisions. For example, it can find whether a new image contains a cat based on the patterns it learned during training.
5. **Feedback and Improvement**: It can improve through feedback, especially in methods like reinforcement learning. In this case, the AI receives rewards or penalties, refining its ability to make better decisions over time.

History: The term **Artificial Intelligence (AI)** is already widely used in everything from smart phones to self-driving cars. AI has come a long way from science fiction stories to practical uses.

R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.



Envision a device with human-like cognitive abilities to learn, think, and solve issues. That is AI's central tenet. AI research aims to create intelligent machines that can replicate human cognitive functions. It has been a long and winding road filled, with moments of tremendous advancement, failures, and moments of reflection.

Fundamentally, Artificial Intelligence is the process of building machines that can replicate human intelligence. These machines can learn, reason, and adapt while carrying out activities that normally call for human intelligence. With artificial intelligence (AI) this world of natural language comprehension, image recognition, and decision making by computers can become a reality.

The Dawn of Artificial Intelligence (1950s-1960s)

The 1950s , which saw the following advancements , are considered to be the birthplace of AI :

- **1950 :** In 1950 saw the publication of Alan Turing's work ,**"Computing Machinery and Intelligence "** which introduced the Turing Test—a measure of computer intelligence.

R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.

- **1956:** A significant turning point in AI research occurs in 1956 when, **John McCarthy** first uses the phrase ***"Artificial Intelligence"*** at the Dartmouth Workshop.
- **1950s–1960s:** The goal of early artificial intelligence (AI) research was to encode human knowledge into computer programs through the use of symbolic reasoning, and logic-based environments.
- **Limited Advancement:** Quick advances are hampered by limited resources and computing-capacity.
- **Early AI systems:** This made an effort to encode human knowledge through the use of logic, and symbolic thinking. The development of early artificial intelligence (AI) systems that, depended on symbolic thinking and logic was hampered by a lack of resources, and processing capacity , which caused the field to advance slowly in the beginning.

AI's Early Achievements and Setbacks (1970s-1980s)

This age has seen notable developments as well as difficulties :

- **1970:** The 1970s witnessed the development of expert systems , which were intended to capture the knowledge of experts in a variety of domains. Data Scientists created rule-based systems that , could use pre-established guidelines to address certain issues.
- **Limitations:** Due to their inability to handle ambiguity and complicated circumstances , these systems had a limited range of applications.
- **The Artificial Intelligence Winter(1970–1980):** A period of inactivity brought on by a lack of funding , and un-met expectations.

Machine Learning and Data-Driven Approaches (1990s)

The 1990s bring a transformative move in AI :

- **1990s:** A worldview move towards machine learning approaches happens.
- **Rise of Machine-Learning:** Calculations learn from information utilizing strategies like neural systems, choice trees , and bolster vector machines.
- **Neural Organize Insurgency:** Propelled by the human brain, neural systems pick up ubiquity for errands like discourse acknowledgment, stock advertise expectation , and motion picture suggestions.
- **Information Powers AI:** Expanded handling control and information accessibility fuel the development of data driven AI.
- **Unused Areas Rise:** Proposal frameworks, picture acknowledgment and normal dialect handling (NLP) take root.

- **Brilliant Age of AI:** AI frameworks exceed expectations in dis-course acknowledgment, stock determining, and suggestion frameworks.
- **Improved Execution:** Handling control enhancements and information accessibility drive progressions.

The AI Boom: Deep Learning and Neural Networks (2000s-2010s)

The 21st century , witnesses the rise of profound learning , and neural systems :

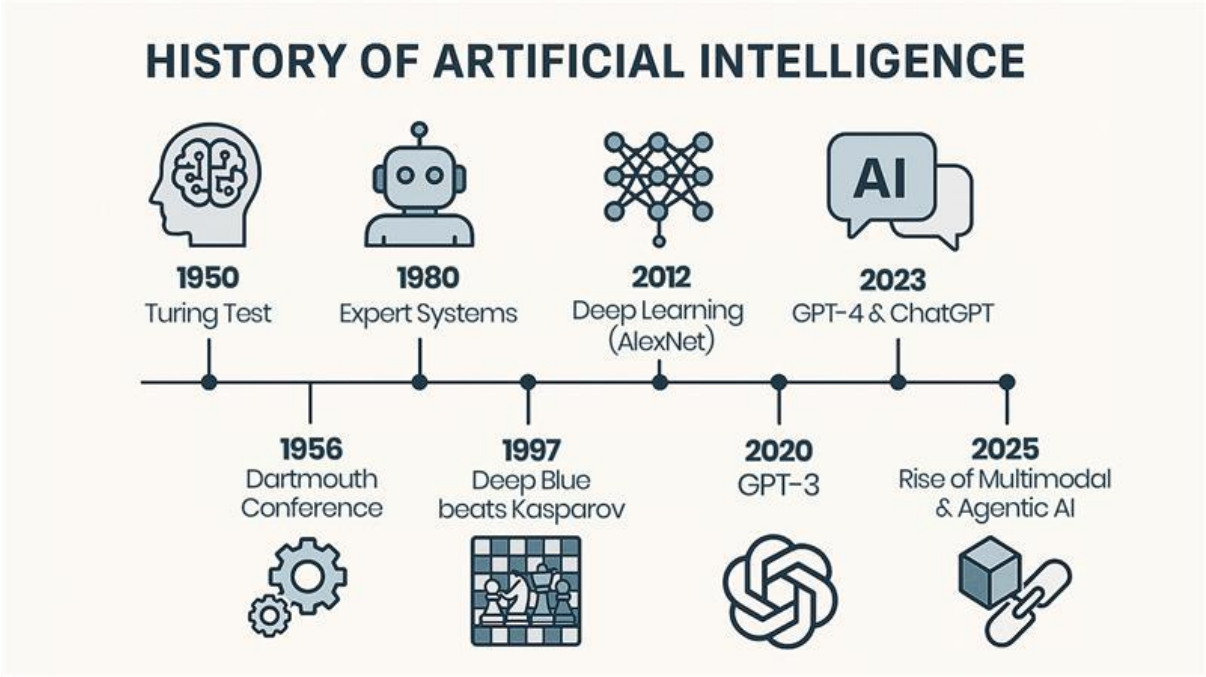
- **2000s-2010s:** Profound learning a subset of machine learning imitating the human brain's structure and work , came to the cutting edge.
- **Profound Neural Systems:** Multi-layered neural systems exceeded expectations in ranges such as - picture acknowledgment, NLP and gaming.
- **Innovative Progressions:** Profound learning encouraged advance in discourse acknowledgment, NLP, and computer vision.
- **Corporate Speculation:** Tech monsters like ***Face book, Google , and OpenAI*** made noteworthy commitments to AI inquire about.
- **Counterfeit Neural Systems:** Complex calculations based on interconnected neurons control profound learning headways.

Generative Pre-trained Transformers: A New Era (GPT Series)

A novel advancement in recent times is the use of Generative Pre-trained Transformers:

- **GPT Series:** Trained on enormous volumes of textual data , these models have rocked the globe.
- **GPT-3:** This model transforms language processing by producing writing that is similar to that of a human being and translating between languages.
- **Learning from Text:** Large volumes of text are absorbed by GPT models, such as - GPT-3, which help them comprehend syntax, context , and comedy.
- **Beyond Translation:** GPT-3 serves as a portable writing assistant by producing essays, poetry , and even language translations.
- **The Upcoming Generation:** This new wave of models , which can write, translate and generate original material as well as provide insightful responses, is exemplified by models such as Bard, ChatGPT, and Bing Copilot.
- **Pushing Boundaries:** These developments have increased the possible applications of AI showcasing its ability in content production, creative projects and language translation.

R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.



Period	Key Events / Achievements	Highlights
Before 1940	Logic & computation theories	Philosophical roots
1940–1956	Turing Test, Neural model	Birth of AI concept
1956–1974	Early AI programs (ELIZA, SHRDLU)	Golden Age
1974–1980	Reduced funding	1st AI Winter
1980–1987	Expert Systems (MYCIN, DENDRAL)	AI revival
1987–1993	Failures of Expert Systems	2nd AI Winter
1993–2011	Machine Learning, Deep Blue	Modern AI begins
2012–Present	Deep Learning, Generative AI	AI Revolution

Current status

- Artificial Intelligence (AI) is now being used widely across various industries and aspects of everyday life.
- With advancements in machine learning, data processing, and automation, AI systems are becoming smarter and more efficient.
- They help organizations make better decisions, improve productivity, and create personalized experiences for users.

R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.

1. Healthcare

AI is transforming healthcare by helping doctors and researchers analyze medical data quickly and accurately.

Some key applications include:

- **Disease Prediction and Diagnosis:** AI models can predict diseases like cancer, diabetes, and heart conditions based on medical history and test data.
- **Medical Image Analysis:** AI systems analyze X-rays, MRIs, and CT scans to detect tumors or fractures faster than humans.
- **Drug Discovery:** AI helps in developing new medicines more efficiently.
- **Virtual Health Assistants:** Chatbots and apps provide health advice and appointment reminders.

2. Finance

AI plays a major role in modern financial systems, helping banks and companies manage large amounts of data securely and efficiently.

- **Fraud Detection:** AI algorithms detect unusual transaction patterns to prevent fraud.
- **Algorithmic Trading:** AI systems make quick and accurate trading decisions based on real-time market data.
- **Credit Scoring:** AI helps evaluate loan applications and assess customer reliability.
- **Customer Service:** Chat bots assist customers with account queries 24/7.

3. Transportation

AI helps make transportation safer, faster, and more reliable.

- **Self-Driving Cars:** Autonomous vehicles use AI for lane detection, obstacle avoidance, and traffic navigation.
- **Smart Traffic Systems:** AI-based systems monitor and control traffic signals to reduce congestion.
- **Route Optimization:** Apps like Google Maps use AI to suggest the fastest routes.
- **Public Transport Scheduling:** AI helps plan efficient bus or train schedules based on passenger demand.

4. Education

AI is changing how students learn and teachers teach by offering customized solutions.

- **Personalized Learning Systems:** AI analyzes each student's progress and adapts lessons accordingly.
- **AI Tutors:** Virtual assistants help students with homework and concepts.

- **Automated Grading:** AI tools can evaluate assignments and exams automatically.
- **Language Translation:** AI helps students access materials in multiple languages.

5. Entertainment

AI makes entertainment more engaging and personalized.

- **Recommendation Systems:** Platforms like Netflix, YouTube, and Spotify use AI to suggest movies, videos, or songs based on your interests.
- **Content Creation:** AI tools generate music, videos, and art.
- **Game Development:** AI improves game characters' intelligence and realism.
- **Film Production:** AI is used in visual effects and editing to enhance creativity.

✓ Artificial Intelligence is now deeply integrated into modern society.

✓ From diagnosing diseases to driving cars and suggesting your next movie, AI

✓ Continues to improve the quality, efficiency, and convenience of life across all fields.

Scope

Field	Application
Robotics	Intelligent robots that interact with humans
Natural Language Processing (NLP)	Speech and text understanding
Computer Vision	Facial and object recognition
Expert Systems	Decision-making support
Machine Learning (ML)	Pattern recognition and prediction
Automation	Smart homes and industrial systems

Agents

Type	Description
Simple Reflex Agent	Acts only on current input (no memory).
Model-Based Reflex Agent	Uses internal memory or model of the world.
Goal-Based Agent	Chooses actions to achieve specific goals.
Utility-Based Agent	Selects actions based on maximum performance measure (happiness, score, etc.).
Learning Agent	Improves its performance with experience.

R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.

Environments

Type	Description
Fully Observable	Agent knows everything about the environment (e.g., Chess).
Partially Observable	Only some information is known (e.g., driving in fog).
Deterministic	The next state is predictable.
Stochastic	Results of actions are uncertain.
Static	Environment doesn't change while the agent is deciding.
Dynamic	Environment changes over time.
Discrete	Finite set of actions or states (e.g., Tic-Tac-Toe).
Continuous	Infinite states (e.g., car driving).

Problem Formulations

Problem formulation means **defining the problem so that an agent can solve it**.

Main Components:

1. **Initial State:** Starting condition.
2. **Actions (Operators):** Possible actions that can be taken.
3. **Transition Model:** Describes what happens after each action.
4. **Goal Test:** Checks if the goal has been reached.
5. **Path Cost:** Evaluates the efficiency of the path (time, cost, etc.).

Example: Route-Finding Problem

- **Initial State:** Starting city
- **Actions:** Move to connected cities
- **Goal Test:** Reached destination city
- **Path Cost:** Distance or time

Review of tree and graph structures

Structure	Definition	Use in AI
Tree	Hierarchical structure with root and branches	Used in *search trees* for exploring solutions
Graph	Collection of nodes connected by edges	Used when states can repeat (loops)

R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.

State space representation

The **state space** is the set of **all possible states** that can be reached in solving a problem.

- **State:** Configuration of the system.
- **Action:** Transition from one state to another.
- **Goal State:** The desired outcome.

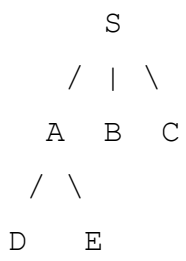
Example:
In an 8-puzzle, every possible arrangement of tiles = a *state*.

Search graph and Search tree

Artificial Intelligence often involves exploring possible states or paths to solve a problem. Depending on how we represent the states and transitions, we use **Search Trees** or **Search Graphs**.

1. Search Tree

- A **search tree** is a tree representation of all possible states generated from an initial state.
- Each node represents a **state** of the problem.
- **Edges** represent actions or transitions from one state to another.
- **Key Features:**
    1. The same state can appear **multiple times** if reached via different paths.
    2. No cycles: Nodes are expanded independently, even if they represent the same state.
    3. Useful for **visualizing the entire search process**.
- **Example:**



    - "S" is the start state.
    - Paths show all possible sequences of moves.

Advantages:

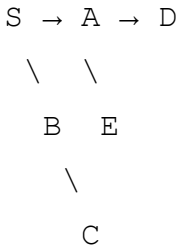
- Simple to implement.
- Easy to track the path taken.

Disadvantages:

- Can be **memory intensive** if states repeat.
- May perform unnecessary work due to repeated exploration.

2. Search Graph

- A **search graph** is a graph representation of all states of a problem, considering each **unique state only once**.
- Nodes = **unique states**, Edges = **actions or transitions**.
- **Key Features:**
  1. Handles **cycles** and avoids visiting the same state repeatedly.
  2. Uses a **closed list** to keep track of visited states.
  3. More **memory efficient** and avoids redundant computations.
- **Example:**



  - Each state is unique; repeated states are merged.

Advantages:

- Avoids revisiting states.
- Suitable for **graph search algorithms** like BFS, DFS with visited checks.

Disadvantages:

- More complex to implement than a tree.
- Needs extra memory to store visited states (closed list).

Comparison Table

Feature	Search Tree	Search Graph
Node representation	Each node = state instance	Each node = unique state
Duplicates	Can have duplicates	No duplicates (unique states)
Cycles	Not handled	Handles cycles
Memory Usage	High for large trees	Efficient with visited list
Use Case	Simple exploration visualization	Efficient problem-solving

- **Search Tree:** Explores every path without checking if a state is repeated.

- **Search Graph:** Avoids repeated states, making it more efficient for large or cyclic state spaces.

UNIT I – Introduction to AI

Two-Mark Questions

1. Define Artificial Intelligence.
2. What are intelligent agents in AI?
3. List any two applications of Artificial Intelligence.
4. What is the Turing Test?
5. Define rational agent.
6. What is state space representation?
7. Differentiate between search tree and search graph.
8. What is the role of the environment in AI?
9. Define problem formulation in AI.
10. What is an uninformed search?

Five-Mark Questions

1. Explain the **scope and applications** of Artificial Intelligence in real life.
2. Describe the **structure of an intelligent agent** with a neat diagram.
3. Explain the **concept of state space representation** with an example.
4. Differentiate between **tree search** and **graph search** in detail.
5. Discuss the **importance of problem formulation** in AI problem solving.

Ten-Mark Questions

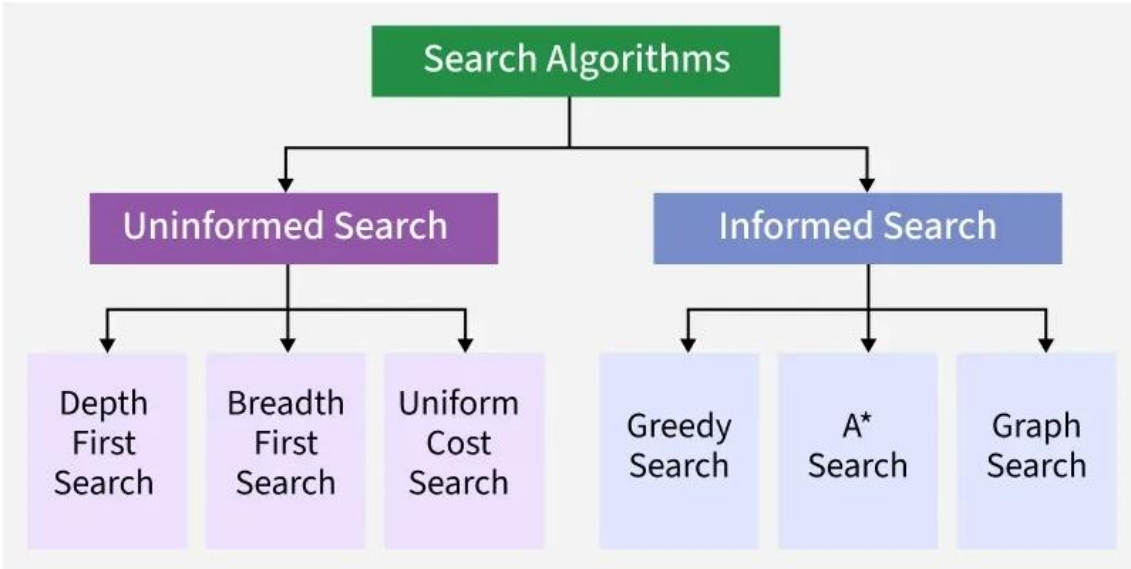
1. Discuss in detail the **history, evolution, and current status of Artificial Intelligence**.
2. Explain the **agent–environment interaction** model. Describe the different types of agents used in AI.
3. Describe **search strategies in AI**. Explain the structure of a **search graph** and a **search tree** with suitable examples and diagrams.

R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.

UNIT II

SEARCH ALGORITHMS

A **Search Algorithm** in Artificial Intelligence is a **method used to find a solution (or path) from a starting state to a goal state** by systematically exploring possible states in a problem space.



There are mainly 2 types of search algorithms i.e Uninformed Search Algorithms and Informed Search Algorithms.

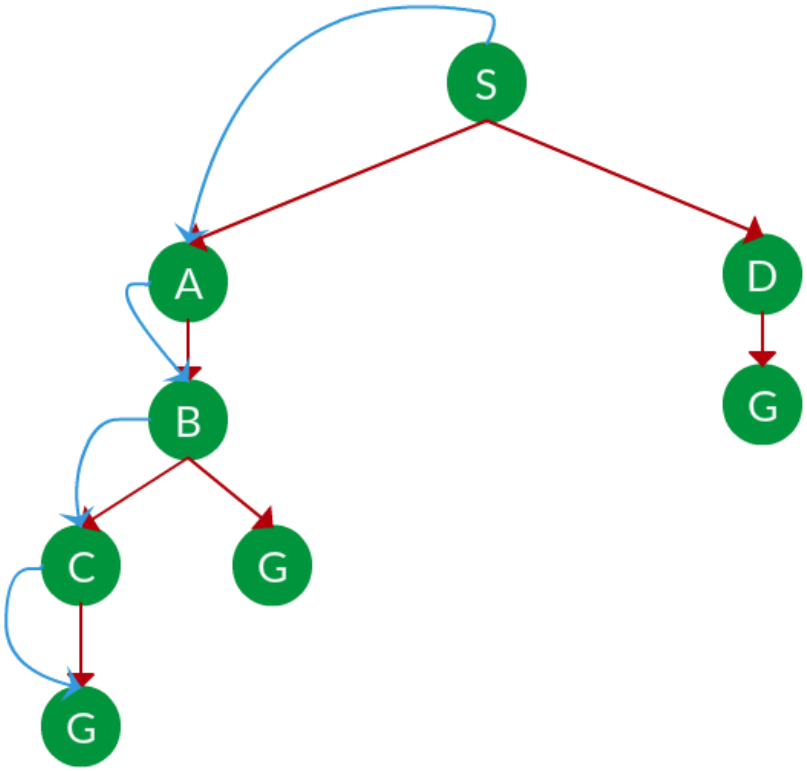
Uninformed Search Algorithms

Uninformed search also called blind search explores the search space without any domain specific knowledge or heuristics. It treats all nodes equally and chooses which path to explore next based solely on general rules like node depth or path cost.

1. Depth First Search

- Depth First Search explores paths by going as deep as possible along one direction before backtracking. It uses a stack or recursion to keep track of the path.
- DFS is memory efficient compared to BFS since it doesn't need to store all siblings at each level.
- However it is not guaranteed to find the shortest path and may get stuck in an infinite loop if the search tree is deep or contains cycles unless depth limits or visited checks are applied.

For Example: Which solution would DFS find to move from node S to node G if run on the graph below.

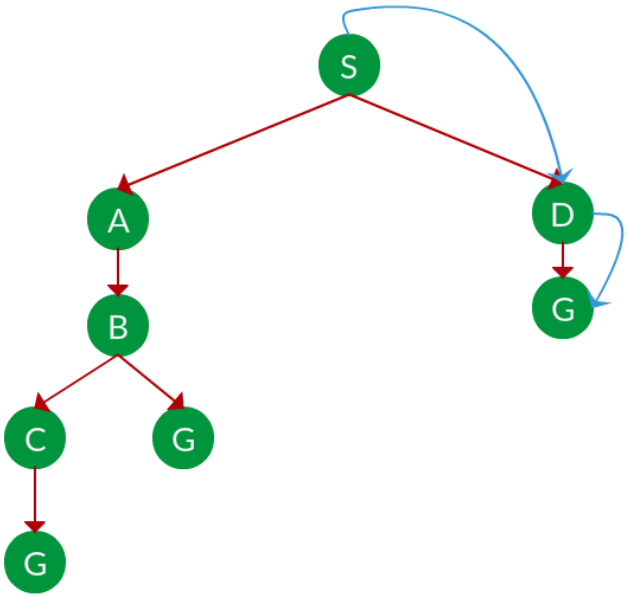


- As DFS traverses the tree deepest node first it would always pick the deeper branch until it reaches the solution or it runs out of nodes and goes to the next branch.
- **Path**: S->A->B->C->G

2. Breadth First Search

- Breadth First Search is a fundamental search algorithm that explores all possible paths level by level. It begins from the root node and explores all neighboring nodes before moving to the next level of nodes.
- BFS is complete and guarantees finding the shortest path if each move has the same cost.
- However its main drawback is high memory usage as it stores all nodes at the current level before moving deeper which can grow rapidly for large or complex problems.

For Example: Which solution would BFS find to move from node S to node G if run on the graph below.

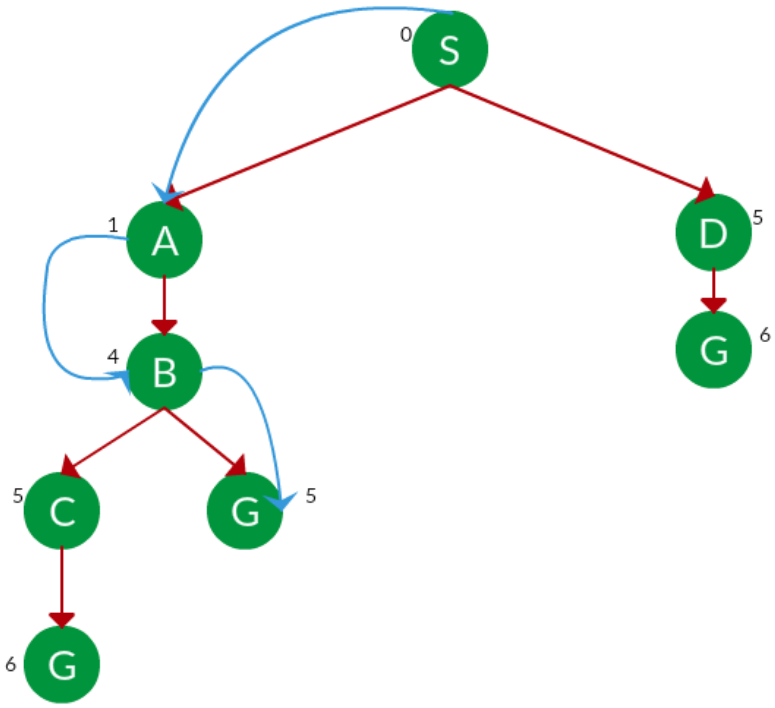


- As BFS traverses the tree shallowest node first it would always pick the shallower branch until it reaches the solution or it runs out of nodes and goes to the next branch.
- **Path**: S->D->G

3. Uniform Cost Search

- Uniform Cost Search is similar to BFS but takes the cost of each move into account. It always expands the node with the lowest cumulative path cost from the start.
- This makes UCS optimal and complete and useful when actions have different costs such as in navigation systems.
- It uses a priority queue to manage the frontier and ensures the cheapest path is always chosen next.

For Example: Which solution would UCS find to move from node S to node G if run on the graph below.



- The cost of each node is the cumulative cost of reaching that node from the root and based on the UCS strategy the path with the least cumulative cost is chosen.
- **Path:** S->A->B->G

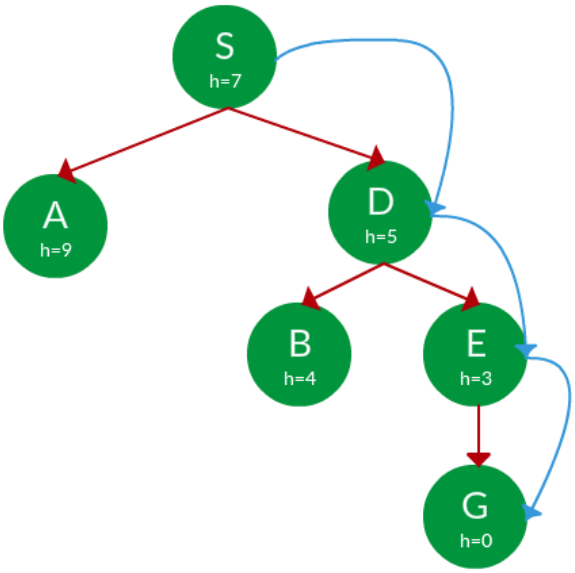
Informed Search Algorithms

Informed search uses domain knowledge in the form of heuristics to make smarter decisions during the search process. These heuristics estimate how close a state is to the goal guiding the search more efficiently.

1. Greedy Search

- Greedy search is a heuristic based algorithm that selects the path which appears to lead most directly toward the goal.
- It makes decisions based solely on the estimated cost from the current node to the goal (h(n)) ignoring the cost already taken to reach the current node (g(n)).
- This approach makes greedy search fast and memory efficient as it focuses only on immediate gains.

For Example: Find the path from S to G using greedy search, the heuristic values h of each node below the name of the node.

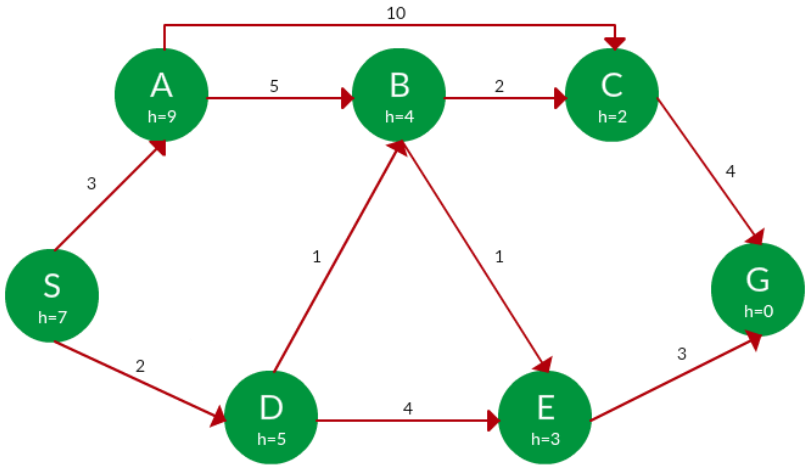


- Starting from S we can traverse to A(h=9) or D(h=5). We choose D as it has the lower heuristic cost.
- Now from D we can move to B(h=4) or E(h=3). We choose E with a lower heuristic cost.
- Finally from E we go to G(h=0). This entire traversal is shown in the search tree below, in blue.
- **Path**: S->D->E->G

2. A* Tree Search:

- A* tree search also uses the f(n) = g(n) + h(n) evaluation but treats the search space as a tree which means it doesn't track already visited nodes. Every path is explored independently even if it leads to the same state.
- This can result in duplicate work and a larger search space, especially in graphs with cycles.
- Although it's simpler to implement A* tree search may be less efficient and is typically used when the search structure is naturally a tree or when memory constraints prevent maintaining a closed list.

For Example: Find the path to reach from S to G using A* search.

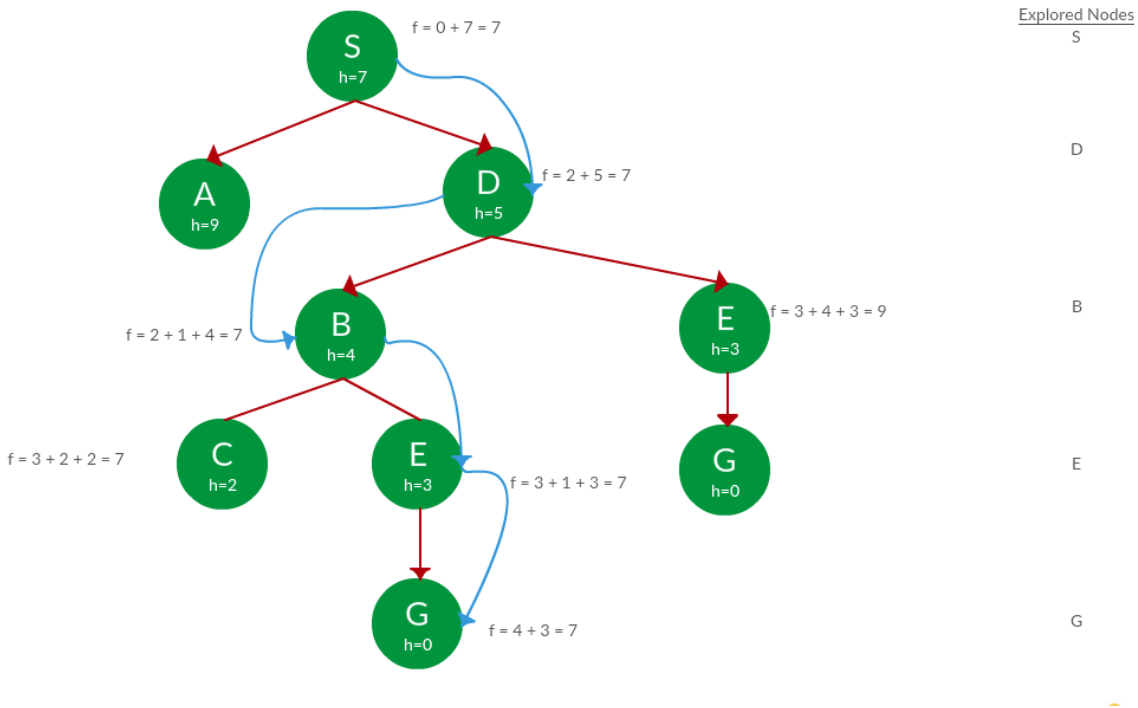


- Starting from S the algorithm computes g(x) + h(x) for all nodes in the fringe at each step choosing the node with the lowest sum.
- **Path**: S->D->B-> G-> E

3. A* Graph search

- A* graph search is an informed algorithm that finds the shortest path in a graph by considering both the cost to reach a node (g(n)) and the estimated cost to the goal (h(n)).
- It keeps track of visited nodes using a closed list to avoid revisiting them which makes it efficient and prevents cycles or redundant paths.
- This approach ensures optimality and completeness if the heuristic is admissible.
- It's suitable for complex environments like road maps or game levels where paths may loop or intersect.

For Example: Use graph searches to find paths from S to G in the following graph.



- In this Algo we keep a track of nodes explored so that we don't re explore them.
- **Path**: S->D->B->E->G

Random Search

Random Search is a **basic search algorithm** that tries to find a solution by **randomly selecting states or solutions** in the search space.
It does **not follow any specific path or rule** for choosing the next state — instead, it randomly explores different possibilities until it finds the goal or a satisfactory result.

Concept:

- Random Search does not use any prior knowledge or heuristics.
- It simply picks **random points** in the search space, checks if they are a solution, and continues until it finds one that works.
- It is mainly used when the **search space is very large or complex**, and no clear strategy is known.

Steps in Random Search Algorithm:

1. **Start** the process.
2. **Generate a random solution** (or state).
3. **Evaluate** if the solution meets the goal or performance criteria.

4. If **goal is reached**, stop and return the solution.
5. If not, **repeat** by generating another random solution.
6. Continue until the goal is found or a limit (like time or number of tries) is reached.

Pseudocode:

```
Random_Search(problem)
{
    while (not goal_state)
    {
        s ← generate_random_state()
        if (goal_test(s))
            return s
    }
}
```

Example:

Suppose we are trying to find the maximum value of a mathematical function:

$f(x) = x^2 – 3x + 2$,

where x ranges between 0 and 5.

- The Random Search will randomly pick values of x (e.g., 1.2, 4.5, 2.8, etc.).
- It computes f(x) for each value.
- It keeps track of the **best value found so far**.
- After several tries, it may find a near-optimal or the best solution.

Advantages:

- Very **simple to implement**.
- **Does not require** gradient information or heuristic knowledge.
- Works well for **non-linear** and **complex problems**.

Disadvantages:

- **Inefficient** — may take a long time to find the solution.
- **No guarantee** of finding the best (optimal) solution.
- **Randomness** can lead to inconsistent results in each run.

Applications:

- Optimization problems

R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.

- Testing algorithms
- Random sampling in simulations
- When no prior knowledge or heuristic is available

Random Search is one of the **simplest AI search methods** that explore the solution space by **randomly generating and testing** possible solutions. Although it is not efficient, it provides a **baseline method** for comparison with more advanced algorithms like **Hill Climbing, Genetic Algorithms, or Simulated Annealing**.

Search with Open and Closed List

- Used in systematic search algorithms (like BFS, DFS, A*).
- **Open List:** Stores nodes that have been generated but not yet explored.
- **Closed List:** Stores nodes that have already been explored.
- Helps avoid repeated states and reduces redundant computation.

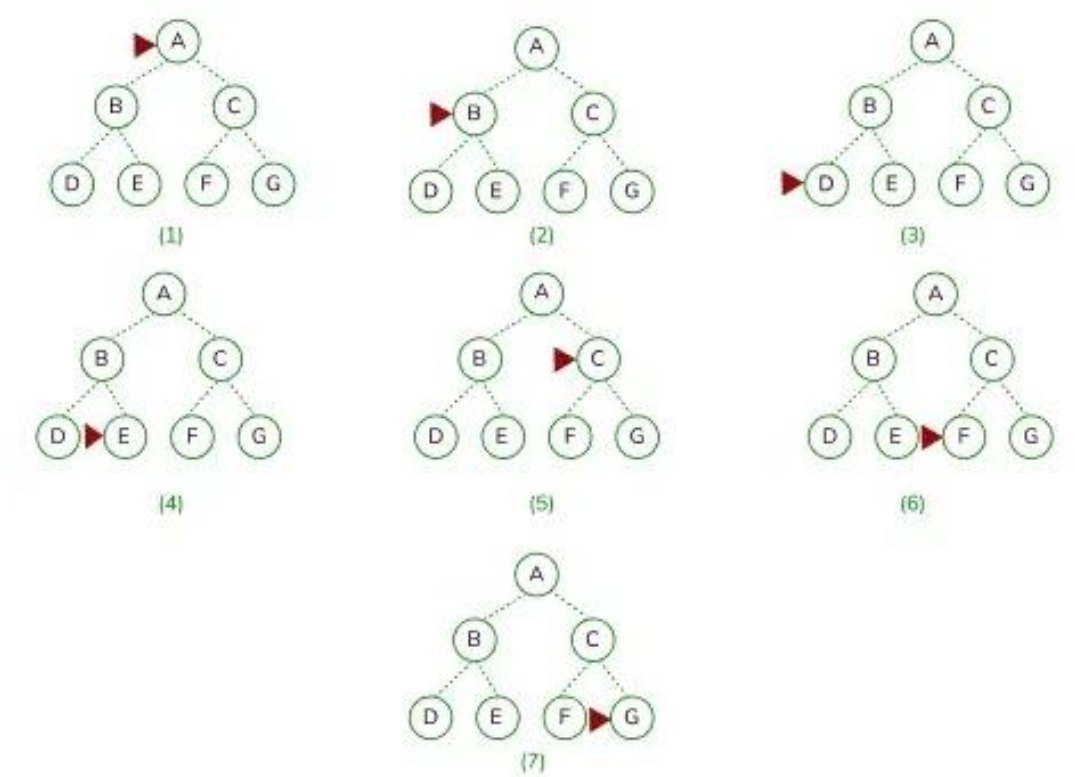
Depth First Search (DFS)

- Explores as far as possible along a branch before backtracking.
- Uses **Stack (LIFO)** structure.
- Suitable for problems with deep but not wide search space.
- **Advantages:** Low memory usage.
- **Disadvantages:** May get stuck in infinite depth, not always optimal.

Depth-first search is a traversing algorithm used in tree and graph-like data structures. It generally starts by exploring the deepest node in the frontier. Starting at the root node, the algorithm proceeds to search to the deepest level of the search tree until nodes with no successors are reached. Suppose the node with unexpanded successors is encountered then the search backtracks to the next deepest node to explore alternative paths.

DFS Working

Depth-first search (DFS) explores a graph by selecting a path and traversing it as deeply as possible before backtracking.



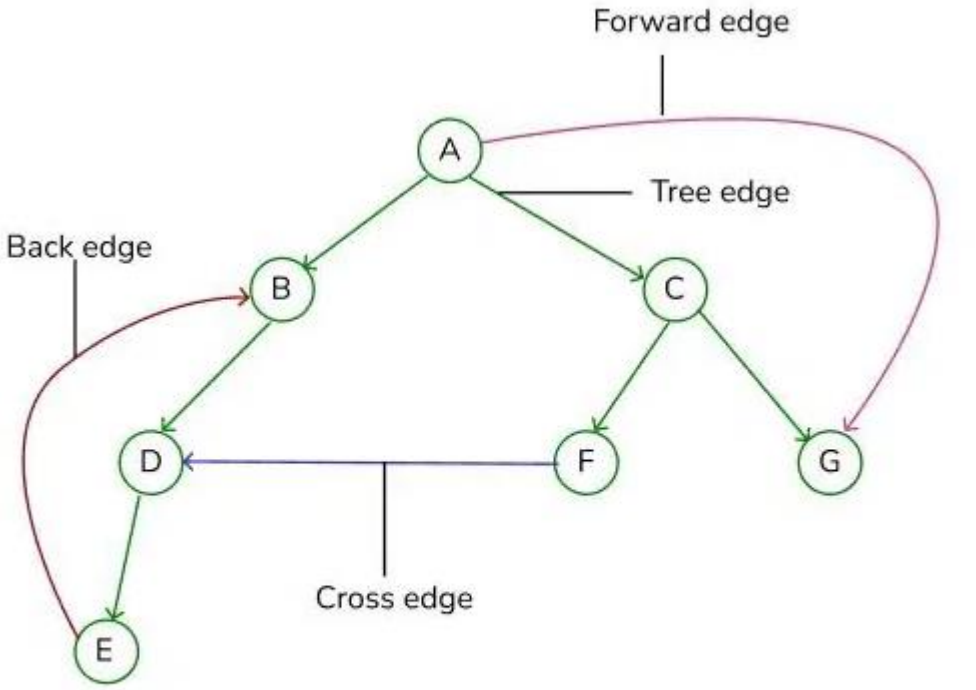
- Originally it starts at the root node, then it expands its one branch until it reaches a dead end. Then it backtracks to the most recent unexplored node, repeating until all nodes are visited or a specific condition is met. (As shown in the above image, starting from node A, DFS explores its successor B, then proceeds to its descendants until reaching a dead end at node D. It then backtracks to node B and explores its remaining successor's i.e E.)
- This systematic exploration continues until all nodes are visited or the search terminates. (In our case after exploring all the nodes of B. DFS explores the right side node i.e C then F and and then G. After exploring the node G. All the nodes are visited. It will terminate.

Key Characteristics of Depth First Search (DFS):

1. **Explores Deeply First** – DFS extends the current path as far as possible before considering alternative paths.
2. **Not Cost-Optimal** – DFS does not guarantee the shortest or cheapest path to the goal. It may find a solution quickly, but it might not be optimal.

3. **Stack-Based Search** – DFS uses a **stack (LIFO)** to keep track of nodes. New nodes are pushed onto the stack, and when a dead end is reached, it backtracks by popping nodes.
4. **Backtracking** – A variant of DFS generates one child at a time and directly modifies the current state, saving memory by storing only the active path.
5. **Low Memory Requirement** – Compared to Breadth First Search, DFS requires less memory since it only stores nodes along the current path.

Edge classes in a Depth-first search tree based on a spanning tree



- **Forward edges:** The forward edge is responsible for pointing from a **node** of the tree to one of its **successors**.
- **Back edges:** The back edge holds the power of directing its edge from a **node** of the tree to one of its **ancestors**.
- **Tree edges:** When DFS explores the new vertex from the current vertex, the edge connecting them is called a tree edge. It is essential while constructing a DFS spanning tree because it represents the paths to be followed during the traversal.

R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.

- **Cross edges:** Cross edges are the edges that connect two vertices that are neither ancestors nor descendants of each other in the DFS tree.

Applications of Depth First Search (DFS) in AI

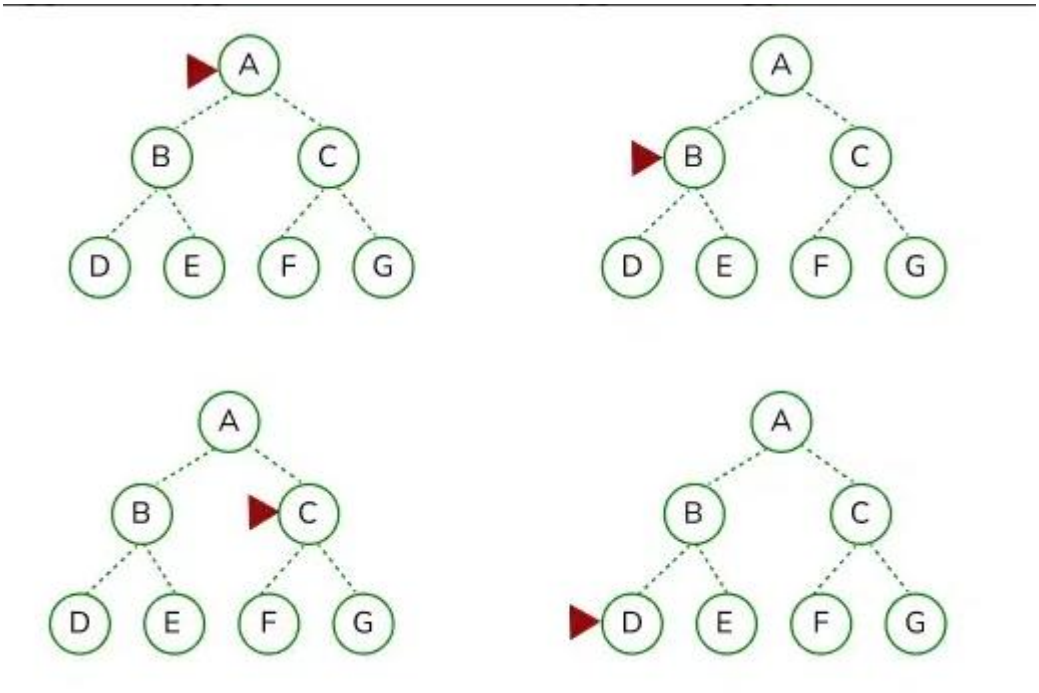
1. **Maze Generation**
    - DFS is used to generate mazes using a **randomized recursive approach**.
    - Works by choosing a random unvisited neighbor, removing the wall between cells, and adding the new cell to the stack.
    - Continues until all cells are visited → results in a complete maze.
2. **Puzzle Solving**
    - Used in solving puzzles like **Japanese nonograms**.
    - DFS explores different combinations of filled and empty cells systematically until a valid solution is found.
3. **Path finding in Robotics**
    - DFS is applied to **path finding problems** where memory efficiency is important.
    - Useful in simple environments where finding *any* valid path (not necessarily the shortest) is acceptable.

Breadth First Search (BFS)

- Explores all nodes at one depth level before moving to the next.
- Uses **Queue (FIFO)** structure.
- Finds the shortest solution if step cost is uniform.
- **Advantages:** Complete and optimal (if all step costs equal).
- **Disadvantages:** High memory usage.

The Breadth-First Search is a traversing algorithm used to satisfy a given property by searching the tree or graph data structure. It belongs to uninformed or blind search AI algorithms as it operates solely based on the connectivity of nodes and doesn't prioritize any particular path over another based on heuristic knowledge or domain-specific information. It doesn't incorporate any additional information beyond the structure of the search space. It is optimal for un weighted graphs and is particularly suitable when all actions have the same cost. Due to its systematic search strategy, BFS can efficiently explore even infinite state spaces.

BFS Working



- Originally it starts at the root node, then it expands to all of its successors It systematically explores all its neighbouring nodes before moving to the next level of nodes. (As shown in the above image, it starts from the root node A and then expands its successor B)
- This process of extending to the root node's immediate neighbours, then to their neighbours, and so on, lasts until all the nodes within the graph have been visited or until the specific condition is met. From the above image we can observe that after visiting the node B it moves to node C. When level 1 is completed, it further moves to the next level i.e 2 and explores node D. Then it will move systematically to node E, node F and node G. After visiting node G, it will terminate.

Key characteristics of BFS

1. **First-in-First-Out (FIFO):** The FIFO queue is preferred in BFS because it is faster than a priority queue and maintains the correct node order. In BFS, new nodes deeper in the tree are added to the back of the queue, while older, shallower nodes are expanded first.

R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.

2. **Early goal test:** In traditional BFS, the algorithm tracks the states reached during the search. Instead of storing all states, it keeps only those needed for an early goal test, checking if a newly generated node satisfies the goal as soon as it's created.
3. **Cost-optimal:** BFS always aims to find a minimum-cost solution by prioritizing the shortest path. When it generates nodes at depth d, it has already explored all nodes at depth $d-1$. So, if a solution exists, BFS will find it as soon as it reaches the correct depth, making it a cost-optimal algorithm.

Applications of Breadth-First Search

1. Path finding Algorithms

- BFS is widely used for finding the **shortest path in unweighted graphs**.
- Explores nodes **level by level**, ensuring the first path found is the shortest.
- **Examples:**
    - Maze solving
    - Robot navigation
    - Route planning (e.g., Google Maps without weights).

2. Web Crawling

- BFS explores web pages **by link depth**.
- Visits all links on a page before moving deeper.
- Ensures **organized and systematic data collection**.
- **Applications:** Search engine indexing, structured data extraction.

3. Social Network Analysis

- Used to analyze **relationships and connections**.
- Helps in:
    - Finding **degrees of separation** (e.g., "friends of friends").
    - Identifying **mutual friends**.
    - Locating **influencers or clusters** in networks.
- Examples: LinkedIn, Facebook.

4. AI in Games

- BFS explores possible moves in **game state trees**.
- **Examples:**
    - Pac-Man: Ghosts use BFS for shortest path chasing.
    - Puzzle games: BFS explores possible moves systematically.
    - Board games: Ensures fair, complete exploration of actions.

Advantages of BFS

1. **Shortest Path Guarantee** – Always finds the shortest path in unweighted graphs.

2. **Simple to Implement** – Uses a **queue (FIFO)**.
3. **Versatile** – Works in many domains (web, games, social networks).
4. **Systematic** – Explores all nodes at one level before moving deeper.

Disadvantages of BFS

1. **High Memory Requirement** – Needs to store all nodes at each level.
2. **Slow for Deep Nodes** – Explores many shallow nodes before reaching deeper ones.
3. **Not Suitable for Weighted Graphs** – Cannot handle edge weights (Dijkstra's or A* is better).

Heuristic Search

- Uses **heuristic (h(n))**, a function that estimates the cost from the current node to the goal.
- Provides guidance to the search instead of blindly exploring.
- Reduces search time and effort.
- Example: 8-puzzle using "number of misplaced tiles" as heuristic.

Heuristic Search Techniques in AI

- Heuristic search techniques are used in AI for **problem-solving and decision-making**.
- They help find the **most efficient path** from a start state to a goal state.
- Unlike brute-force methods, heuristic search uses **heuristic information** (rules of thumb/estimates) to guide the search.
- Balances:
    - **Exploration** – trying new possibilities.
    - **Exploitation** – refining known good solutions.

Significance of Heuristic Search

- Efficiently navigates **large search spaces**.
- Reduces number of nodes explored → saves **time and computation**.
- Enables AI to solve **complex problems** that exact algorithms cannot.

Components of Heuristic Search

1. **State Space** – All possible states of the problem.
2. **Initial State** – The starting state.
3. **Goal Test** – Condition to check if the goal has been reached.
4. **Successor Function** – Defines possible next states (moves).
5. **Heuristic Function (h(n))** – Estimates cost or distance from current state to goal.

R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.

Types of Heuristic Search Techniques

1. *A Search Algorithm**
    - Evaluation function:

      $f(n)=g(n)+h(n)f(n) = g(n) + h(n)$
        - $g(n)$ = cost so far
        - $h(n)$ = estimated cost to goal
    - **Complete, Optimal, Efficient** (if heuristic is admissible).
    - Used in pathfinding, navigation, robotics.
2. **Greedy Best-First Search**
    - Chooses node with the **lowest h(n)** (closest to goal).
    - Fast but not always optimal.
    - Example: Route finding.
3. **Hill Climbing**
    - Iteratively moves to the best neighboring state (local improvement).
    - Can get stuck in **local maxima** or plateaus.
    - Used in optimization problems.
4. **Simulated Annealing**
    - Probabilistic search inspired by metallurgy.
    - Accepts worse solutions temporarily to escape local maxima.
    - Gradually reduces randomness → converges to global optimum.
5. **Beam Search**
    - Explores only **k best nodes (beam width)** at each level.
    - Saves memory compared to BFS.
    - Useful in speech recognition, machine translation.

Applications of Heuristic Search

1. **Path finding** – GPS navigation, robotics, game maps.
2. **Optimization** – Scheduling, resource allocation.
3. **Game Playing** – Chess, Go, strategy planning.
4. **Robotics** – Obstacle avoidance, task planning.
5. **Natural Language Processing (NLP)** – Parsing, semantic analysis, text generation.

Advantages

1. **Efficiency** – Reduces unnecessary exploration.
2. **Optimality** – Algorithms like A* guarantee best solution (with admissible heuristic).
3. **Versatility** – Applicable across many fields (games, robotics, NLP, planning).

R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.

Limitations

1. **Heuristic Quality** – Poor heuristics → slow or wrong results.
2. **Space Complexity** – Some need large memory (e.g., A*).
3. **Domain-Specific** – Designing heuristics requires domain knowledge.

Best First Search

- Expands the node with the **best heuristic value** (lowest estimated cost).
- Uses a **priority queue**.
- Greedy approach → chooses node closest to the goal.
- **Advantages:** Faster than BFS/DFS for many problems.
- **Disadvantages:** May not be optimal, depends on heuristic quality.

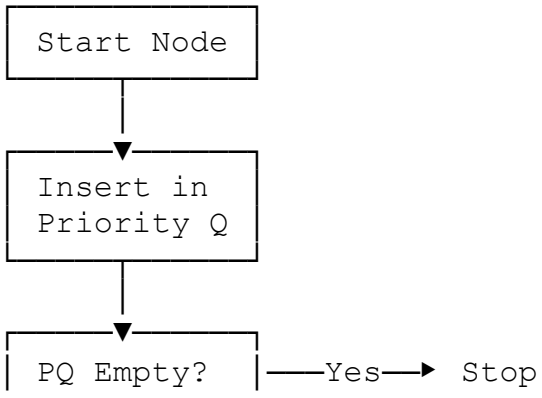
Best First Search (Heuristic Search)

Best First Search is a **graph traversal/search algorithm** that explores a graph by always expanding the **most promising node** chosen according to a heuristic evaluation function (often based on edge cost or estimated distance to goal).

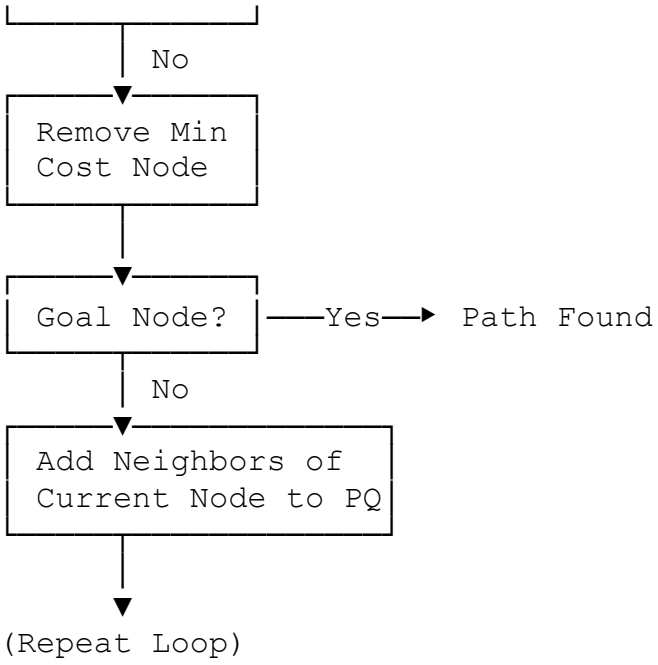
Algorithm (Step-by-Step)

1. Initialize a **priority queue (PQ)** and insert the starting node with cost `0`.
2. Mark the start node as **visited**.
3. While PQ is not empty:
    - Remove the node `u` with the **lowest heuristic value** from PQ.
    - If `u` is the **goal node**, stop the search (path found).
    - Otherwise, for each neighbor `v` of `u`:
        - If `v` is not visited:
            - Mark `v` as visited.
            - Insert `v` into PQ with its heuristic (edge cost).
4. Continue until goal is reached or PQ becomes empty.

Flowchart (Steps Representation)



R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.



Python Implementation

```
import heapq
from collections import defaultdict

def best_first_search(edges, source, target):
    # Build adjacency list
    graph = defaultdict(list)
    for u, v, w in edges:
        graph[u].append((w, v))
        graph[v].append((w, u))  # undirected graph

    visited = set()
    pq = [(0, source, [source])]  # (cost, node, path)

    while pq:
        cost, node, path = heapq.heappop(pq)

        if node == target:
            return path

        if node not in visited:
            visited.add(node)
            for w, neighbor in graph[node]:
                if neighbor not in visited:
                    heapq.heappush(pq, (w, neighbor, path +
[neighbor]))

    return None
```

R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.

Example Run

```
edges = [
    [0, 1, 3], [0, 2, 6], [0, 3, 5], [1, 4, 9],
    [1, 5, 8], [2, 6, 12], [2, 7, 14], [3, 8, 7],
    [8, 9, 5], [8, 10, 6], [9, 11, 1], [9, 12, 10], [9, 13, 2]
]

print("Path (0 → 9):", best_first_search(edges, 0, 9))
print("Path (0 → 8):", best_first_search(edges, 0, 8))
```

Output

```
Path (0 → 9): [0, 1, 3, 2, 8, 9]
Path (0 → 8): [0, 1, 3, 2, 8]
```

Key Points for Exam:

- Best First Search uses **priority queue**.
- Always selects node with **minimum heuristic cost**.
- Efficient for **path finding, optimization, AI navigation**.
- Limitation: May not guarantee optimal solution (depends on heuristic).

A Algorithm*

- A popular informed search algorithm.
- Uses **evaluation function:**

  $f(n)=g(n)+h(n)f(n) = g(n) + h(n)$

  - g(n) = cost from start to current node
  - h(n) = estimated cost from current node to goal
- Properties:
  - **Complete** (finds a solution if one exists).
  - **Optimal** (finds the best solution if heuristic is admissible).
  - **Efficient** (less time than uninformed searches).

A* Search Algorithm

Definition

A* (A-star) is a **heuristic search algorithm** used in Artificial Intelligence to find the **shortest path** from a **start node** to a **goal node**.

It combines:

- **Uniform Cost Search (UCS)** (which considers path cost so far)
- **Greedy Best-First Search (which uses heuristic estimate)**

Evaluation Function

A* uses an evaluation function:

f(n)=g(n)+h(n)f(n) = g(n) + h(n)

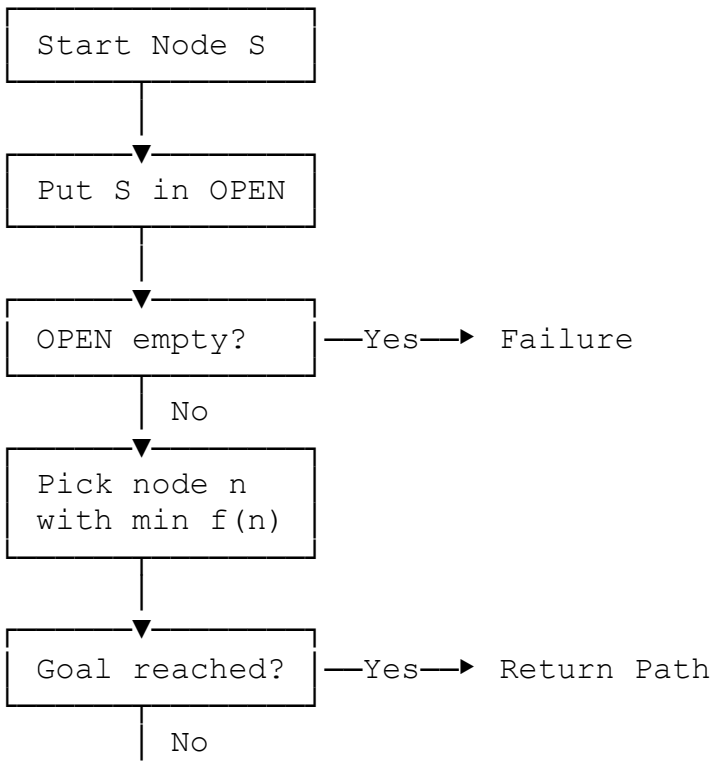
Where:

- **g(n)** = actual cost from start node to current node n.
- **h(n)** = estimated cost from node n to the goal (heuristic function).
- **f(n)** = total estimated cost of the path through n.

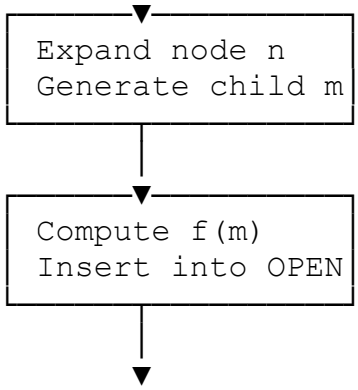
Algorithm (Step-by-Step)

1. Initialize an **open list (priority queue)** and insert the **start node** with cost f(start) = g(start) + h(start).
2. Initialize an **empty closed list** (visited nodes).
3. While the open list is not empty:
    - Remove the node n with the **lowest f(n)** from the open list.
    - If n is the **goal node**, return the path (done).
    - Otherwise, expand n:
        - For each neighbor m of n:
            - Calculate g(m) = g(n) + cost(n, m)
            - Calculate f(m) = g(m) + h(m)
            - If m is not in open/closed list, insert it in the open list.
    - Move n to the **closed list**.
4. If open list becomes empty → no solution exists.

Flowchart (Step Representation)



R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.



Python Implementation

```
import heapq

def a_star(graph, start, goal, h):
    open_list = []
    heapq.heappush(open_list, (0 + h[start], 0, start,
[start]))  # (f, g, node, path)
    closed_set = set()

    while open_list:
        f, g, node, path = heapq.heappop(open_list)

        if node == goal:
            return path, g  # path and total cost

        if node in closed_set:
            continue

        closed_set.add(node)

        for neighbor, cost in graph[node]:
            if neighbor not in closed_set:
                g_new = g + cost
                f_new = g_new + h[neighbor]
                heapq.heappush(open_list, (f_new, g_new,
neighbor, path + [neighbor]))

    return None, float("inf")
```

Example Run

```
graph = {
    'A': [('B', 1), ('C', 3)],
    'B': [('D', 3), ('E', 1)],
    'C': [('F', 5)],
    'D': [('G', 2)],
    'E': [('G', 1)],
```

R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.

```
    'F': [('G', 2)],
    'G': []
}

# Heuristic values (straight-line estimate to goal G)
h = {'A': 7, 'B': 6, 'C': 4, 'D': 2, 'E': 1, 'F': 3, 'G': 0}

path, cost = a_star(graph, 'A', 'G', h)
print("Optimal Path:", path)
print("Total Cost:", cost)
```

Output

```
Optimal Path: ['A', 'B', 'E', 'G']
Total Cost: 3
```

Key Points

- **Admissible heuristic** (never overestimates) → A* guarantees optimal solution.
- **Efficient** compared to BFS/DFS/UCS.
- **Widely used in pathfinding, robotics, games (e.g., Google Maps, Chess AI).**
- **Limitation:** Needs good heuristic, otherwise can be memory-heavy.

Game Search

- Used in AI for **two-player competitive games** (Chess, Tic-Tac-Toe).
- Represented using a **Game Tree**.
- **Minimax Algorithm:**
  - Maximizing player tries to maximize score.
  - Minimizing player tries to minimize score.
- **Alpha-Beta Pruning:** Technique to reduce number of nodes evaluated by pruning unnecessary branches.

Game Search in AI

- Game Search is a search technique in Artificial Intelligence used in **two-player games** (e.g., Chess, Tic-Tac-Toe, Checkers).
- The idea is to simulate all possible moves of both players (search tree) and select the **optimal move** using algorithms such as **Minimax** and **Alpha-Beta Pruning**.

1. Game Tree

- A **game tree** is a tree where:
  - Each **node** = a game state.
  - Each **edge** = a possible move.

- Levels alternate between **MAX player** (tries to maximize score) and **MIN player** (opponent, tries to minimize score).

2. Minimax Algorithm

Concept

- Used in **zero-sum games** (one player's gain = another's loss).
- Assumes:
  - **MAX player** → wants to maximize the score.
  - **MIN player** → wants to minimize the score.

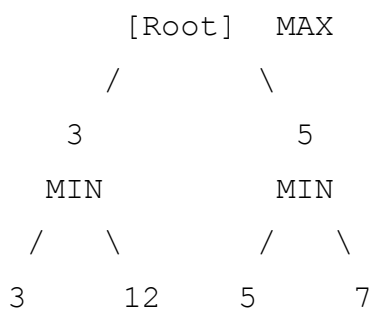
Steps

1. Generate the game tree up to a certain depth.
2. Apply the **evaluation function** at leaf nodes.
3. Propagate values upward:
   - At **MAX nodes** → choose maximum child value.
   - At **MIN nodes** → choose minimum child value.
4. Root's value decides the **best move**.

3. Alpha-Beta Pruning

- Improvement over Minimax.
- **Prunes (cuts off)** branches of the game tree that cannot influence the final decision.
- Uses two values:
  - **α (alpha):** Best value MAX can guarantee so far.
  - **β (beta):** Best value MIN can guarantee so far.
- If **α ≥ β**, stop exploring that branch.

4. Example (Tic-Tac-Toe Mini Tree)



- MAX chooses min(3,12) = 3 from left branch.
- MAX chooses min(5,7) = 5 from right branch.
- Finally, MAX selects **max(3,5) = 5**.

  Best move for MAX is 5.

5. Applications of Game Search

R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.

- Board games: Chess, Checkers, Tic-Tac-Toe.
- Puzzle-solving games.
- Decision-making in strategy-based video games.
- AI agents in competitive environments.

Advantages

- Provides **optimal moves** if search tree is fully explored.
- Applicable to a wide range of two-player games.

Disadvantages

- Computationally expensive for large games (like Chess).
- Needs **pruning (Alpha-Beta)** to reduce complexity.

UNIT II – Search Algorithms

Two-Mark Questions

1. What is a search algorithm in Artificial Intelligence?
2. Define random search.
3. What is the difference between open list and closed list in search algorithms?
4. What is Depth First Search (DFS)?
5. What is Breadth First Search (BFS)?
6. Define heuristic function.
7. What is meant by heuristic search?
8. What is the main idea behind Best First Search?
9. Define A* algorithm.
10. What is game search in AI?

Five-Mark Questions

1. Explain the **difference between Depth First Search and Breadth First Search** with examples.
2. Describe the **working of heuristic search algorithms** with a suitable example.
3. Explain **search with open and closed lists** and their role in pathfinding.
4. Discuss the **Best First Search algorithm** and write its pseudocode.
5. Explain the **concept of game search** and the role of the minimax algorithm.

Ten-Mark Questions

1. Explain in detail the **A* algorithm**. Describe how it combines the advantages of uniform cost and heuristic search.
2. Compare and contrast **uninformed search** (Random, DFS, BFS) and **informed search** (Heuristic, Best First, A*) algorithms with examples.
3. Discuss **Game Search** in Artificial Intelligence. Explain the **Minimax algorithm** and **Alpha-Beta pruning** with a neat diagram and example.

R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.

UNIT III

Probabilistic Reasoning:

- In Artificial Intelligence, **probabilistic reasoning** is used when there is **uncertainty** in the data or the environment.
- It helps an AI system make decisions even when it doesn't have complete or perfect information.

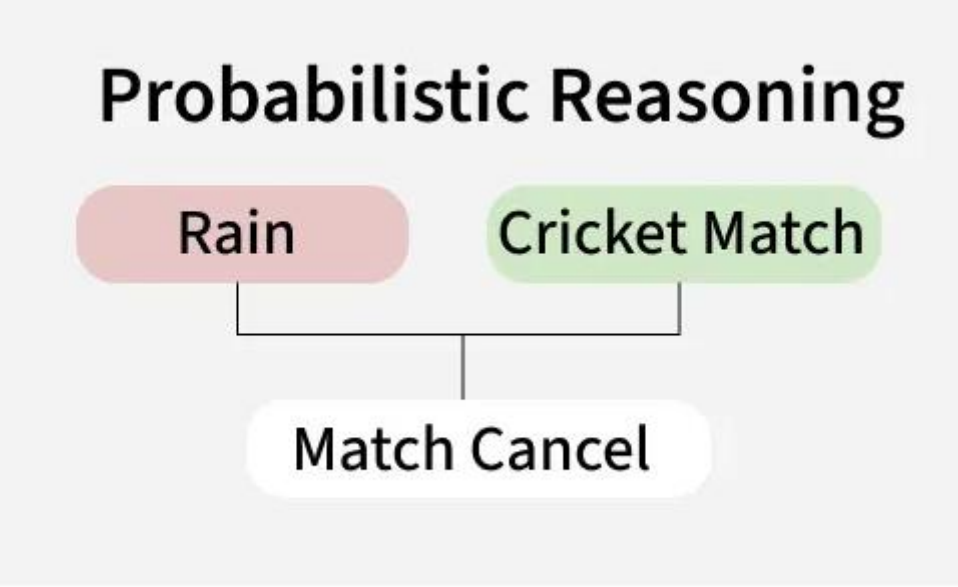
Example:

If a patient has a fever, cough, and sore throat, probabilistic reasoning helps calculate the likelihood that they have the flu.

Probability, conditional probability

Probabilistic reasoning in Artificial Intelligence (AI) is a method that uses probability theory to manage and model uncertainty in decision-making. Unlike traditional systems that depend on precise information, it understands that real-world data is often incomplete, unclear or noisy.

By giving probabilities to different possibilities, AI systems can make better decisions, predict outcomes and solve problems even when things are uncertain. This approach is important for building smart systems that can work in changing environments and make good choices in complex situations.



Probabilistic Reasoning

Need for Probabilistic Reasoning in AI

Probabilistic reasoning with artificial intelligence is important to different tasks such as:

- **Machine Learning:** It helps algorithms learn from incomplete or noisy data, refining predictions over time.
- **Robotics**: It enables robots to navigate and interact with dynamic and unpredictable environments.
- **Natural Language Processing**: It helps AI understand human language which is often ambiguous and depends on context.
- **Decision Making**: It allows AI systems to evaluate different outcomes and make better decisions by considering the likelihood of various possibilities.

Key Concepts in Probabilistic Reasoning

Probabilistic reasoning helps AI systems make decisions and predictions when they have to deal with uncertainty. It uses different ideas and models to understand how likely things are even when we don't have all the answers. Let's see some of the important concepts:

1. Probability: It is a way to measure how likely something is to happen, typically expressed as a number between 0 and 1. In AI, we use probabilities to understand and make predictions when the information we have is uncertain or incomplete.

2. Bayes' Theorem: It helps AI systems update their beliefs when they get new information. It's like changing our mind about something based on new evidence. This is useful when we need to adjust our predictions after learning new facts.

$P(A/B)=P(B/A)\cdot P(A)P(B)P(A/B)=P(B)P(B/A)\cdot P(A)$

Where:

- P(A|B)$P(A|B)$: Probability of A happening, given B has happened (posterior probability).
- P(B|A)$P(B|A)$: Probability of B happening, given A happened.
- P(A)$P(A)$: Prior probability of A.
- P(B)$P(B)$: Probability of observing B.

3. Conditional Probability: It is the chance of an event happening, given that something else has already happened. This helps when the outcome depends on something that happened before.

4. Random Variables: They are values that can change or vary based on uncertainty. In simple terms, these are the things AI tries to predict or estimate. The possible outcomes of these variables depend on probability.

Types of Probabilistic Models

There are different models that use these concepts to help AI systems make sense of uncertainty. Let's see some of them:

1. **Bayesian Networks:** They are graphs that show how different variables are connected with probabilities. Each node represents a variable and the edges show how they depend on each other. These networks help us understand how one piece of information can affect another.
2. **Markov Models:** They predict the future state of a system based only on the present state with no regard for the past. This is known as the "memoryless" property which means the future depends only on the current situation not the history that led to it.
3. **Hidden Markov Models (HMMs):** They extend Markov models by introducing hidden states that cannot be directly observed. These models help infer the hidden states based on observable data, using statistical techniques to estimate the likelihood of these unobservable conditions.
4. **Probabilistic Graphical Models:** It combine the features of Bayesian networks and Hidden Markov Models, allowing more complex relationships between variables to be represented. It provide a framework for managing uncertainty in large systems where many variables are connected and interact with each other.
5. **Markov Decision Processes (MDPs):** They are used for decision-making, particularly in reinforcement learning. It model an agent's interaction with an environment where the agent takes actions that affect the state of the environment and receives rewards or penalties based on those actions.

Techniques in Probabilistic Reasoning

1. **Inference**: It calculates the probability of an outcome based on known data. Exact methods like variable elimination and approximate methods like Markov Chain Monte Carlo (MCMC) are used, depending on the complexity of the system.
2. **Learning**: It updates the parameters of probabilistic models as new data comes in, improving predictions. Techniques like maximum likelihood estimation and Bayesian estimation allow models to adapt and become more accurate over time.
3. **Decision Making**: AI uses probabilistic reasoning to make decisions that maximize expected rewards. Partially Observable Markov Decision Processes (POMDPs) are used when some information is hidden.

How Probabilistic Reasoning Enhances AI Systems?

Probabilistic reasoning helps AI systems navigate uncertainty, enabling them to make better decisions even when information is unclear. Let's see how it works:

1. **Quantifying Uncertainty**: Probabilistic reasoning turns uncertainty into probabilities. Instead of a simple "yes" or "no," it provides a probability, like "there's a 60% chance of rain tomorrow."
2. **Reasoning with Evidence**: As new information comes in, AI systems update their predictions. For example, if dark clouds appear, the chance of rain might rise to 80%. This continuous adjustment helps AI stay accurate.
3. **Learning from Past Experiences**: AI systems can improve predictions by learning from historical data. For example, weather predictions become more accurate as the AI system accounts for past seasonal trends.
4. **Effective Decision-Making**: It allows AI to make informed decisions by considering the likelihood of different outcomes. It helps AI weigh possible paths and choose the best option even when the future is uncertain.

Applications of Probabilistic Reasoning in AI

Probabilistic reasoning is applicable in a variety of domains which includes:

1. **Robotics:** In robotics, probabilistic reasoning helps with navigation and mapping. For example, in Simultaneous Localization and Mapping, robots create maps and track their position in unknown environments.
2. **Healthcare**: AI systems use probabilistic models to predict disease likelihood and assist in diagnosis. Bayesian networks can model medical factors like symptoms and test results to guide decisions.
3. **Natural Language Processing**: In tasks like speech recognition and translation, models like Hidden Markov Models (HMMs) help AI understand and process ambiguous language.
4. **Finance**: Probabilistic reasoning in finance helps predict market trends and assess risks. Techniques like Bayesian inference and Monte Carlo simulations model financial uncertainties for better decision-making.

Advantages of Probabilistic Reasoning

- **Flexibility**: Probabilistic models can handle different kinds of uncertainty and can be adapted to work in various fields from healthcare to robotics.
- **Robustness**: These models remain effective even when the data is noisy or incomplete, making them reliable in real-world scenarios where perfect data is rarely available.
- **Transparency**: It provides a clear framework for understanding and explaining uncertainty which helps build trust and improve the interpretability of AI decisions.

- **Scalability**: It can scale to handle large amounts of data and complex systems, making them suitable for applications like big data analysis and large-scale decision-making processes.
- **Decision Support**: These models assist in making informed decisions under uncertainty by calculating the likelihood of different outcomes, helping AI systems choose the best course of action based on expected results.

Challenges of Probabilistic Reasoning

Despite its various advantages, probabilistic reasoning in AI also has several challenges:

1. **Complexity**: Some models such as large Bayesian networks can become computationally expensive, especially as the number of variables grows. This can slow down processing and limit scalability.
2. **Data Quality**: Probabilistic models heavily rely on accurate and clean data. If the data is noisy, incomplete or biased, the model's predictions can become unreliable, leading to incorrect conclusions.
3. **Interpretability**: Understanding how probabilistic models make decisions can be tough, particularly in complex systems or deep learning models. This makes it harder to trust and explain AI decisions to non-experts.

Bayes Rule

- **Bayes' Theorem** is a mathematical formula used to determine the **conditional probability** of an event based on prior knowledge and new evidence.
- It adjusts probabilities when new information comes in and helps make better decisions in uncertain situations.

Bayes Theorem and Conditional Probability

- Bayes' theorem (also known as the Bayes Rule or Bayes Law) is used to determine the conditional probability of event A when event B has already occurred.
- The general statement of Bayes' theorem is "The conditional probability of an event A, given the occurrence of another event B, is equal to the product of the probability of B, given A, and the probability of A divided by the probability of event B.**"** i.e.

For example, *if we want to find the probability that a white marble drawn at random came from the first bag, given that a white marble has already been drawn, and there are three bags each containing some white and black marbles, then we can use Bayes' Theorem.*

Bayes Theorem Formula

For any two events A and B, **Bayes's** formula for the Bayes theorem is given by:

Bayes Theorem Formula

For any two events A and B, **Bayes's** formula for the Bayes theorem is given by:

$$P(A|B) = \frac{P(B|A)\,P(A)}{P(B)}$$

- **P(A)** and **P(B)** are the probabilities of events A and B; also, P(B) is never equal to zero.
- **P(A|B)** is the probability of event A when event B happens,
- **P(B|A)** is the probability of event B when A happens.

Bayes Theorem Applications

Bayesian inference is very important and has found application in various activities, including medicine, science, philosophy, engineering, sports, law, etc., and Bayesian inference is directly derived from Bayes theorem.

Some of the Key Applications are:

- **AI & Machine Learning** → Used in **Naïve Bayes classifiers** to predict outcomes.
- **Medical Testing** → finding the real probability of having a disease after a positive test.
- **Spam Filters** → checking if an email is spam based on keywords.
- **Weather Prediction** → updating the chance of rain based on new data.

Difference between Conditional Probability and Bayes Theorem

The difference between Conditional Probability and Bayes's theorem can be understood with the help of the table given below.

Bayes Theorem	Conditional Probability
Bayes's Theorem is derived using the definition of conditional probability. It is used to find the reverse probability.	Conditional Probability is the probability of event A when event B has already occurred.
Formula: P(A|B) = [P(B|A)P(A)] / P(B)	**Formula:** P(A|B) = P(A∩B) / P(B)
Purpose: To update the probability of an event based on new evidence.	**Purpose**: To find the probability of one event based on the occurrence of another.
Focus: Uses prior knowledge and evidence to compute a revised probability.	**Focus**: Direct relationship between two events.

R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.

Solved Examples of Bayes' Theorem

Example 1: A person has undertaken a job. The probability of completing the job on time if it rains is 0.44, and the probability of completing the job on time if it does not rain is 0.95. If the probability that it will rain is 0.45, then determine the probability that the job will be completed on time.

Let:

- *R: event that it rains*
- *RcRc: event that it does not rain*
- *C: event that the job is completed on time*

We are given:

$P(R)=0.45, P(Rc)=1-0.45=0.55 P(R)=0.45, P(Rc)=1-0.45=0.55$

$P(C/R)=0.44, P(C/Rc)=0.95 P(C/R)=0.44, P(C/Rc)=0.95$

By the law of total probability:

$P(C)=P(R)P(C/R)+P(Rc)P(C/Rc) P(C)=P(R)P(C/R)+P(Rc)P(C/Rc)$

Substitute values:

$P(C)=(0.45)(0.44)+(0.55)(0.95) P(C)=(0.45)(0.44)+(0.55)(0.95)$

$P(C)=0.198+0.5225=0.7205 P(C)=0.198+0.5225=0.7205$

Bayesian Networks- representation, construction and inference.

- A **Bayesian Network** (also called a **Belief Network**) is a **graph-based model** that represents relationships between different variables using probabilities.
- It helps us **predict unknown values** and **reason under uncertainty**.

1. Components of a Bayesian Network

Nodes

- Each node represents a **random variable**.
- The variable can be **discrete** (e.g., "Rain = Yes/No") or **continuous** (e.g., temperature).

Edges

- **Directed edges (arrows)** show how one variable **influences** another.
- If node **A** affects node **B**, an arrow is drawn from **A → B**.
- This means **B** depends on **A**.

Example:

```
Cloudy → Rain → WetGrass
```

Here, *Rain* depends on *Cloudy*, and *WetGrass* depends on *Rain*.

R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.

2. Conditional Independence

The main idea behind Bayesian networks is **conditional independence**.

It means:

Each node is **independent of all other nodes** in the network **except its parents**.

➡ In simple terms:

Once you know a node's **parents**, you don't need to know anything else to predict that node.

Example:

If you know whether it rained, the information about whether it was cloudy doesn't add extra help in predicting if the grass is wet.

---

3. Joint Probability Distribution (JPD)

- A **Bayesian network** represents the **joint probability distribution** of all variables in a simpler form.
- Instead of writing one big probability for all variables, we **break it down** into smaller conditional probabilities.

[

P(X_1, X_2, …, X_n) = \prod_{i=1}^{n} P(X_i | Parents(X_i))

]

This formula means:

The overall probability is found by multiplying the probability of each variable given its parents.

Example:

For the network

`Cloudy → Rain → WetGrass`

The joint probability is:

[

P(Cloudy, Rain, WetGrass) = P(Cloudy) × P(Rain | Cloudy) × P(WetGrass | Rain)

]

This **factorization** makes computations easier and faster.

---

4. Inference in Bayesian Networks

Inference means using known information (evidence) to find unknown probabilities.

R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.

Types of Inference:

1. **Exact Inference** – Gives accurate results
    - Example algorithms:
        - **Variable Elimination**
        - **Junction Tree Algorithm**
2. **Approximate Inference** – Used for large networks, gives close results
    - Example techniques:
        - **Monte Carlo Sampling**
        - **Loopy Belief Propagation**

Example:

If you know the grass is wet, you can use inference to calculate how likely it is that it rained.

---

5. Learning in Bayesian Networks

Building (or learning) a Bayesian network involves **two steps**:

(a) Structure Learning

- Finding the best **graph structure (DAG)** that connects variables.
- Algorithms search for a structure that best fits the data.

(b) Parameter Learning

- Once the structure is fixed, we find the **conditional probability values**.
- Methods used:
    - **Maximum Likelihood Estimation (MLE)** – estimates probabilities directly from data.
    - **Bayesian Estimation** – includes prior beliefs to adjust probabilities.

Temporal model

- **Temporal models** are used in Artificial Intelligence to represent how things **change over time**.
- They help in modeling **dynamic systems**, where the current state depends on previous states.

In simple words —

Temporal models capture how the **past influences the present** and how the **present affects the future**.

- Example:

  Weather today influences the weather tomorrow.

For example:

If it is **rainy today**, it is **likely to be cloudy or rainy tomorrow**.

So, each **time step** (today, tomorrow, etc.) is connected to the **next** through probabilities.

Two main types:

1. **Hidden Markov Models (HMM)**

- A **Hidden Markov Model** is a type of temporal model where:
  - The **system's true state** is **hidden** (not directly observable).
  - We can only **observe signals or evidence** that depend on the hidden state.
- It assumes that:
  - The **current state** depends only on the **previous state** (Markov property).
  - The **observations** depend only on the **current hidden state**.

Example:

Speech recognition —

- The **spoken word (hidden state)** cannot be seen,
- But we can hear **sound waves (observations)** to guess what was said.

Structure:

```
Hidden States:    Weather_t → Weather_t+1 → Weather_t+2
Observations:     RainSound_t   RainSound_t+1   RainSound_t+2
```

2. **Dynamic Bayesian Networks (DBN)**

- A **Dynamic Bayesian Network** is a **generalization of HMM**.
- It represents **complex systems** with **multiple variables** that change over time.
- Each time step is modeled as a **Bayesian Network**, and these networks are connected across time.

Example:

In a medical monitoring system:

- A person's **heart rate**, **oxygen level**, and **blood pressure** today affect their health tomorrow.

Structure:

```
Time t:          Health_t → SensorReading_t
                      ↓
Time t+1:      Health_t+1 → SensorReading_t+1
```

R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.

Hidden Markov model

When working with sequences of data, we often face situations **where we can't directly see the important factors that influence the datasets**. **Hidden Markov Models (HMM)** help solve this problem by predicting these hidden factors based on the observable data

Hidden Markov Model in Machine Learning

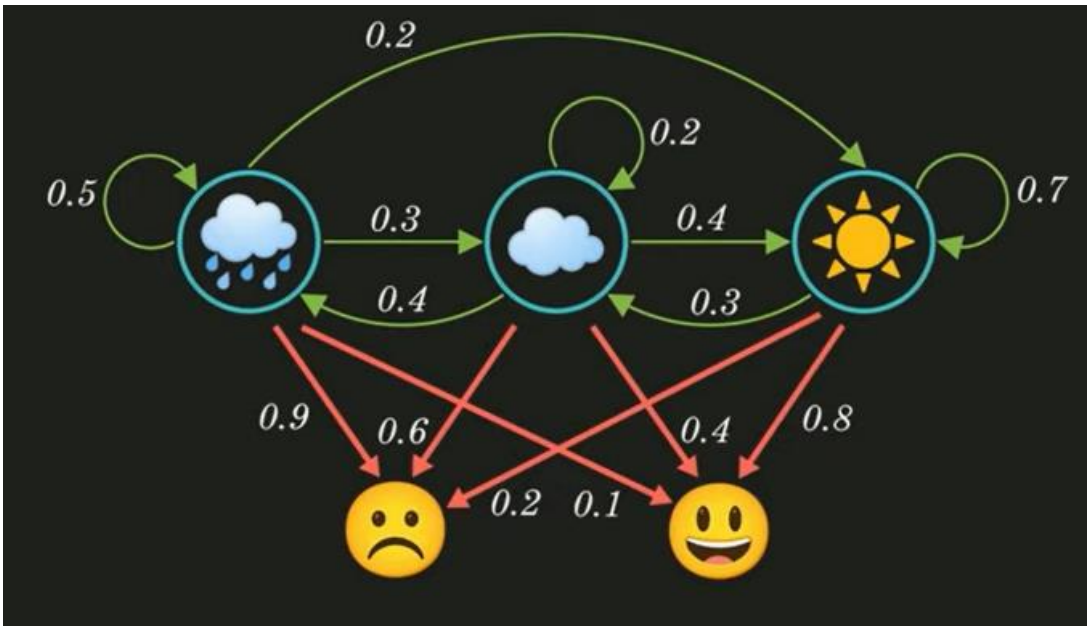
It is an statistical model that is used to describe the **probabilistic relationship between a sequence of observations and a sequence of hidden states**. Iike it is often used in situations where the underlying system or process that generates the observations is unknown or hidden, hence it has the name "**Hidden Markov Model**."

An HMM consists of two types of variables: hidden states and observations.

- The **hidden states** are the underlying variables that generate the observed data, but they are not directly observable.
- The **observations** are the variables that are measured and observed.

The relationship between the hidden states and the observations is modeled using a probability distribution. The Hidden Markov Model (HMM) is the relationship between the hidden states and the observations using two sets of probabilities: the transition probabilities and the emission probabilities.

- The **transition probabilities** describe the probability of transitioning from one hidden state to another.
- The **emission probabilities** describe the probability of observing an output given a hidden state.



R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.

Hidden Markov Model Algorithm

The Hidden Markov Model (HMM) algorithm can be implemented using the following steps:

- **Step 1: Define the state space and observation space:** The state space is the set of all possible hidden states, and the observation space is the set of all possible observations.
- **Step 2: Define the initial state distribution:** This is the probability distribution over the initial state.
- **Step 3: Define the state transition probabilities:** These are the probabilities of transitioning from one state to another. This forms the transition matrix, which describes the probability of moving from one state to another.
- **Step 4: Define the observation likelihoods:** These are the probabilities of generating each observation from each state. This forms the emission matrix, which describes the probability of generating each observation from each state.
- **Step 5: Train the model:** The parameters of the state transition probabilities and the observation likelihoods are estimated using the Baum-Welch algorithm, or the forward-backward algorithm. This is done by iteratively updating the parameters until convergence.
- **Step 6: Decode the most likely sequence of hidden states:** Given the observed data, the Viterbi algorithm is used to compute the most likely sequence of hidden states. This can be used to predict future observations, classify sequences, or detect patterns in sequential data.
- **Step 7: Evaluate the model:** The performance of the HMM can be evaluated using various metrics, such as accuracy, precision, recall, or F1 score.

To summarise, the HMM algorithm involves defining the state space, observation space, and the parameters of the state transition probabilities and observation likelihoods, training the model using the Baum-Welch algorithm or the forward-backward algorithm, decoding the most likely sequence of hidden states using the Viterbi algorithm, and evaluating the performance of the model.

Implementation of HMM in python

Till now we have covered the essential steps of HMM and now lets move towards the hands on code implementation of the following

Key steps in the Python implementation of a simple Hidden Markov Model (HMM) using the **hmmlearn library.**

Applications of HMM

- **Speech Recognition**
- **Bioinformatics (gene prediction)**
- **Finance (market trend prediction)**
- **Natural Language Processing (POS tagging)**
- **Gesture Recognition**

UNIT III – Probabilistic Reasoning

Two-Mark Questions

1. Define probability.
2. What is conditional probability?
3. State Bayes' Rule.
4. What is a Bayesian Network?
5. Define node and edge in Bayesian Networks.
6. What is meant by inference in Bayesian networks?
7. Define temporal model.
8. What is a Hidden Markov Model (HMM)?
9. What are the components of HMM?
10. Write the difference between prior and posterior probability.

Five-Mark Questions

1. Explain Bayes' Rule with a suitable example.
2. Describe the construction of Bayesian Networks with a neat diagram.
3. Discuss how inference is performed in Bayesian Networks.
4. Explain the importance of temporal models in AI.
5. Explain the working and applications of Hidden Markov Models.

Ten-Mark Questions

1. Explain in detail **Bayesian Networks**—their representation, construction, and inference process.
2. Derive **Bayes' Theorem** and explain its significance in probabilistic reasoning.
3. Discuss the structure and working of a **Hidden Markov Model (HMM)** with an example.

R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.

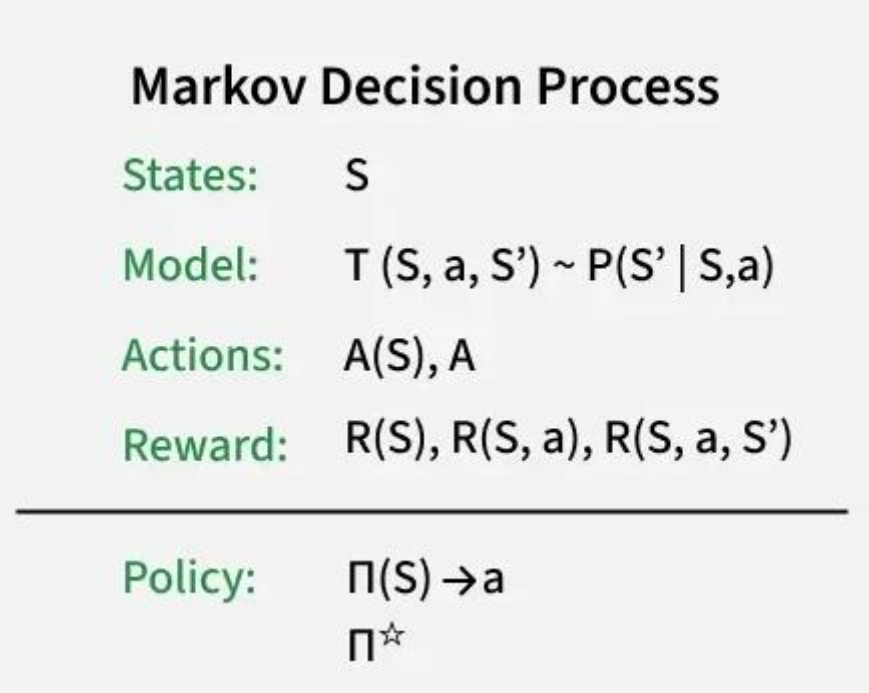
UNIT IV

Markov Decision process

- Markov Decision Process (MDP) is a way to describe how a decision-making agent like a robot or game character moves through different situations while trying to achieve a goal.
- MDPs rely on variables such as the environment, agent's actions and rewards to decide the system's next optimal action.
- In artificial intelligence Markov Decision Processes (MDPs) are used to model situations where decisions are made one after another and the results of actions are uncertain.
- They help in designing smart machines or agents that need to work in environments where each action might led to different outcomes.

Key Components of an MDP

An MDP has five main parts:



Components of Markov Decision Process

1. States (S): A state is a situation or condition the agent can be in. For example, A position on a grid like being at cell (1,1).

2. Actions (A): An action is something the agent can do. For example, Move UP, DOWN, LEFT or RIGHT. Each state can have one or more possible actions.

R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.

3. Transition Model (T): The model tells us what happens when an action is taken in a state. It's like asking: "If I move RIGHT from here, where will I land?" Sometimes the outcome isn't always the same that's uncertainty. For example:

- 80% chance of moving in the intended direction
- 10% chance of slipping to the left
- 10% chance of slipping to the right

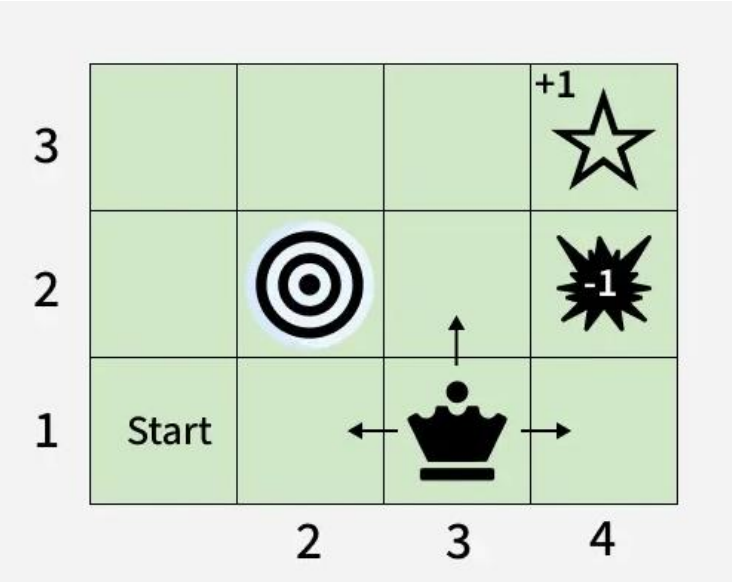
This randomness is called a stochastic transition.

4. Reward (R): A reward is a number given to the agent after it takes an action. If the reward is positive, it means the result of the action was good. If the reward is negative it means the outcome was bad or there was a penalty help the agent learn what's good or bad. Examples:

- +1 for reaching the goal
- -1 for stepping into fire
- -0.1 for each step to encourage fewer moves

5. Policy (π): A policy is the agent's plan. It tells the agent: "If you are in this state, take this action." The goal is to find the best policy that helps the agent earn the highest total reward over time.

Let's consider a 3x4 grid world. The agent starts at cell (1,1) and aims to reach the Blue Diamond at (4,3) while avoiding Fire at (4,2) and a Wall at (2,2). At each state the agent can take one of the following actions: UP, DOWN, LEFT or RIGHT



Problem

1. Movement with Uncertainty (Transition Model)

The agent's moves are stochastic (uncertain):

- 80% chance of going in the intended direction.
- 10% chance of going left of the intended direction.
- 10% chance of going right of the intended direction.

2. Reward System

- +1 for reaching the goal.
- -1 for falling into fire.
- -0.04 for each regular move (to encourage shorter paths).
- 0 for hitting a wall (no movement or penalty).

3. Goal and Policy

- The agent's objective is to maximize total rewards.
- It must find an optimal policy: the best action to take in each state to reach the goal quickly while avoiding danger.

4. Path Example

- One possible optimal path is: UP → UP → RIGHT → RIGHT → RIGHT
- But because of randomness the agent must plan carefully to avoid accidentally slipping into fire.

Applications

Markov Decision Processes are useful in many real-life situations where decisions must be made step-by-step under uncertainty. Here are some applications:

- **Robots and Machines:** Robots use MDPs to decide how to move safely and efficiently in places like factories or warehouses and avoid obstacles.
- **Game Strategy:** In board games or video games MDPs help characters to choose the best moves to win or complete tasks even when outcomes are not certain.
- **Healthcare:** Doctors can use it to plan treatments for patients, choosing actions that improve health while considering uncertain effects.
- **Traffic and Navigation:** Self-driving cars or delivery vehicles use it to find safe routes and avoid accidents on unpredictable roads.
- **Inventory Management:** Stores and warehouses use MDPs to decide when to order more stock so they don't run out or keep too much even when demand changes.

MDP formulation

An **MDP** is formally represented as a 5-tuple:

[
MDP = (S, A, P, R, \gamma)
]

Term	Description
S	Set of all possible **states**. Example: robot's position in a grid.
A	Set of all possible **actions**. Example: move left, right, up, down.
****P(s'**	s, a)**
R(s, a)	**Reward function** — immediate reward received after performing action `a` in state `s`.
γ (gamma)	**Discount factor (0 ≤ γ < 1)** — represents how much future rewards are valued compared to immediate rewards.

Example: Grid World

- **States (S):** each cell in the grid.
- **Actions (A):** {Up, Down, Left, Right}.
- **Reward (R):** +10 for reaching goal, -1 for every step.
- **Transition (P):** 0.8 probability move succeeds, 0.2 random direction.
- **Discount (γ):** 0.9.

Goal: Find an **optimal policy (π^*)** that maximizes the expected total reward.

Utility theory

- The **utility** of a state represents how desirable it is for the agent to be in that state.
- The **utility function** helps in comparing different states and deciding which action to take.

Expected Utility:

[
U(s) = E \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right]
]
Where:

- (U(s)): utility (expected total reward) of state `s`.
- (\gamma^t): discounting future rewards.
- (R(s_t, a_t)): reward received at time `t`.

Utility functions

Utility function (U): Assigns a numeric value to each state.

R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.

- Represents **how good** it is to be in that state, considering future rewards.

Two key ideas:

1. **Additive rewards:** Total utility = sum of discounted rewards.
2. **Expected utility:** Incorporates uncertainty in transitions.

Value iteration

Value Iteration computes the **optimal utility values** for each state iteratively, and then derives the optimal policy.

Pseudocode:

```
1. Initialize U(s) = 0 for all states s
2. Repeat until convergence:
       For each state s:
           U(s) ← R(s) + γ * max over a [ Σs' P(s'|s,a) *
U(s') ]
3. Return U(s)
4. Derive policy:
       π*(s) = argmax over a [ Σs' P(s'|s,a) * U(s') ]
```

Key Idea:

Each iteration updates the utility of a state based on:

- The **reward** for the current state.
- The **expected utility** of the next possible states.

Policy iteration

Policy Iteration alternates between:

1. **Policy Evaluation** – Compute utility values for a fixed policy.
2. **Policy Improvement** – Update the policy using current utility values.

Pseudocode:

```
1. Initialize a random policy π(s)
2. Repeat:
       (a) Policy Evaluation:
           Solve Uπ(s) = R(s) + γ Σs' P(s'|s, π(s)) * Uπ(s')
       (b) Policy Improvement:
           π(s) ← argmax over a [ Σs' P(s'|s,a) * Uπ(s') ]
3. Until policy π converges
```

Comparison with Value Iteration

Feature	Value Iteration	Policy Iteration
Focus	Updates utilities directly	Updates policies explicitly
Speed	Faster per iteration	May take fewer iterations overall
Convergence	Gradual	Often faster overall

R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.

Partially observable MDPs

In some real-world scenarios, the agent **cannot fully observe the state of the environment**. A **Partially Observable Markov Decision Process (POMDP)** extends MDP to handle uncertainty in observations.

POMDP Components

[
POMDP = (S, A, P, R, O, Z, \gamma)
]

Symbol	Description
S	States
A	Actions
P	Transition probabilities
R	Rewards
O	Observations
**Z(o	s’,a)**
γ	Discount factor

In POMDPs, the agent maintains a **belief state** (a probability distribution over possible states) instead of knowing the exact state.

Example of POMDP

A **robot navigation system** where:

- Sensors are noisy.
- It may not know exactly where it is but can estimate the probability of being in each location.
- It uses that **belief** to choose the best next action.

Applications of MDP

- **Robotics:** Navigation and motion planning.
- **Autonomous Vehicles:** Path decision under uncertainty.
- **Finance:** Optimal investment strategies.
- **Operations Research:** Resource allocation.
- **Game AI:** Strategy optimization.

R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.

UNIT IV – Markov Decision Process (MDP)

Two-Mark Questions

1. What is a Markov Decision Process (MDP)?
2. Define the term “state” in MDP.
3. What is a policy in MDP?
4. Define reward function.
5. What is utility in MDP?
6. Define value iteration.
7. Define policy iteration.
8. What is the difference between deterministic and stochastic transitions?
9. What is a partially observable MDP (POMDP)?
10. List any two applications of MDP.

Five-Mark Questions

1. Explain the **components of MDP** with an example.
2. Describe the **utility theory and utility function** in decision making.
3. Explain the **value iteration** process in MDP.
4. Discuss the **policy iteration** method with an example.
5. Explain **partially observable MDPs (POMDPs)** and their importance.

Ten-Mark Questions

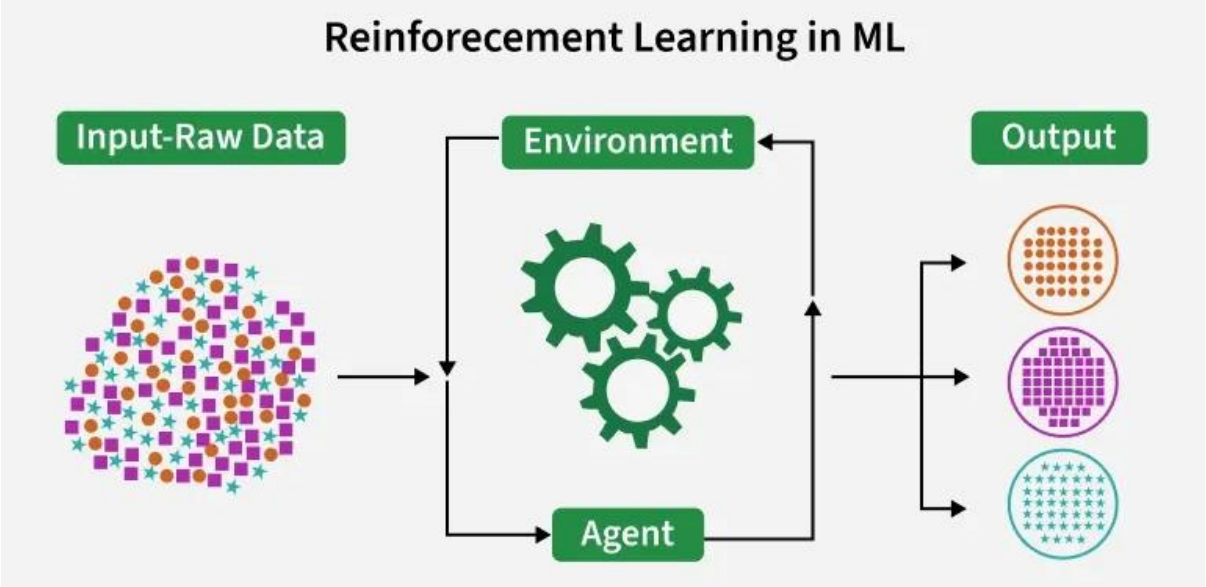
1. Explain **Markov Decision Process (MDP)** formulation with all components and a suitable example.
2. Compare **value iteration** and **policy iteration** in detail.
3. Discuss **utility theory and partially observable MDPs** with applications.

R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.

UNIT V

Reinforcement Learning:

Reinforcement Learning (RL) is a branch of machine learning that focuses on how agents can learn to make decisions through trial and error to maximize cumulative rewards. RL allows machines to learn by interacting with an environment and receiving feedback based on their actions. This feedback comes in the form of rewards or penalties.



Reinforcement Learning revolves around the idea that an agent (the learner or decision-maker) interacts with an environment to achieve a goal. The agent performs actions and receives feedback to optimize its decision-making over time.

- **Agent**: The decision-maker that performs actions.
- **Environment**: The world or system in which the agent operates.
- **State**: The situation or condition the agent is currently in.
- **Action**: The possible moves or decisions the agent can make.
- **Reward**: The feedback or result from the environment based on the agent's action.

Core Components

Let's see the core components of Reinforcement Learning

1. Policy

- Defines the agent's behavior i.e maps states for actions.
- Can be simple rules or complex computations.
- **Example**: An autonomous car maps pedestrian detection to make necessary stops.

2. Reward Signal

- Represents the goal of the RL problem.

R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.

- Guides the agent by providing feedback (positive/negative rewards).
- **Example**: For self-driving cars rewards can be fewer collisions, shorter travel time, lane discipline.

3. Value Function

- Evaluates long-term benefits, not just immediate rewards.
- Measures desirability of a state considering future outcomes.
- **Example**: A vehicle may avoid reckless maneuvers (short-term gain) to maximize overall safety and efficiency.

4. Model

- Simulates the environment to predict outcomes of actions.
- Enables planning and foresight.
- **Example**: Predicting other vehicles' movements to plan safer routes.

Working of Reinforcement Learning

The agent interacts iteratively with its environment in a feedback loop:

- The agent observes the current state of the environment.
- It chooses and performs an action based on its policy.
- The environment responds by transitioning to a new state and providing a reward (or penalty).
- The agent updates its knowledge (policy, value function) based on the reward received and the new state.
- This cycle repeats with the agent balancing exploration (trying new actions) and exploitation (using known good actions) to maximize the cumulative reward over time.

This process is mathematically framed as a Markov Decision Process (MDP) where future states depend only on the current state and action, not on the prior sequence of events.

Types of Reinforcements

1. Positive Reinforcement: Positive Reinforcement is defined as when an event, occurs due to a particular behavior, increases the strength and the frequency of the behavior. In other words, it has a positive effect on behavior.

- **Advantages**: Maximizes performance, helps sustain change over time.
- **Disadvantages**: Overuse can lead to excess states that may reduce effectiveness.

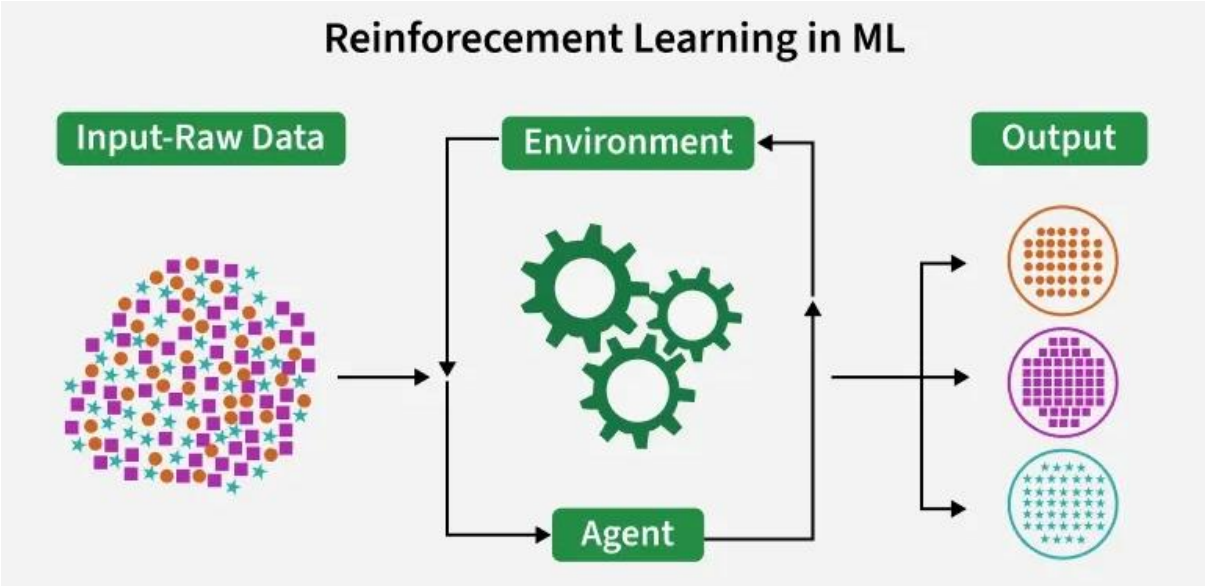
2. Negative Reinforcement: Negative Reinforcement is defined as strengthening of behavior because a negative condition is stopped or avoided.

- **Advantages**: Increases behavior frequency, ensures a minimum performance standard.

- **Disadvantages**: It may only encourage just enough action to avoid penalties.

Online vs. Offline Learning

Reinforcement Learning can be categorized based on how and when the learning agent acquires data from its environment, dividing the methods into online RL and offline RL (also known as batch RL).



Online vs Offline RL

- In online RL, the agent learns by actively interacting with the environment in real-time. It collects fresh data during training by executing actions and observing immediate feedback as it learns.
- Offline RL trains the agent exclusively on a pre-collected static dataset of interactions generated by other agents, human demonstrations or historical logs. The agent does not interact with the environment during learning.

Application

- **Robotics**: RL is used to automate tasks in structured environments such as manufacturing, where robots learn to optimize movements and improve efficiency.
- **Games**: Advanced RL algorithms have been used to develop strategies for complex games like chess, Go and video games, outperforming human players in many instances.
- **Industrial Control**: RL helps in real-time adjustments and optimization of industrial operations, such as refining processes in the oil and gas industry.
- **Personalized Training Systems**: RL enables the customization of instructional content based on an individual's learning patterns, improving engagement and effectiveness.

Advantages

- Solves complex sequential decision problems where other approaches fail.
- Learns from real-time interaction, enabling adaptation to changing environments.
- Does not require labeled data, unlike supervised learning.
- Can innovate by discovering new strategies beyond human intuition.
- Handles uncertainty and stochastic environments effectively.

Disadvantages

- Computationally intensive, requiring large amounts of data and processing power.
- Reward function design is critical; poor design leads to unintended behaviors.
- Not suitable for simple problems where traditional methods are more efficient.
- Challenging to debug and interpret, making it hard to explain decisions.
- Exploration-exploitation trade-off requires careful balancing to optimize learning.

Passive reinforcement learning

- **Passive Reinforcement Learning (Passive RL)** is a type of reinforcement learning where the **agent follows a fixed policy** — that is, it already knows what action to take in each state — and its goal is to **learn how good that policy is**.
- In simple terms:

  The agent **does not try to find a better policy**, it only **evaluates** the one it's given.

In Passive RL:

- The agent observes what happens when it follows the given policy.
- It records the rewards it receives.
- Then it **learns the utility (value)** of each state — how good it is to be in that state.

This helps the agent **understand how effective the current policy** is at achieving the goal.

Example

Imagine a **robot moving in a grid world**:

- Each cell is a **state**.
- The robot has a **policy** (for example: always move right until you hit a wall, then move up).
- The robot doesn't change this behavior; it just follows it and **learns** the total reward it gets from each cell.

Over many runs, the robot estimates:

- Which cells are closer to the goal (high reward)
- Which cells are risky or lead to penalties (low reward)

Approaches to Passive RL

There are **three main methods** used in passive reinforcement learning:

1. ***Direct Utility Estimation***
2. ***Adaptive Dynamic Programming (ADP)***
3. ***Temporal Difference (TD) Learning***

Applications

- Evaluating existing strategies in **robot control**.
- Learning the effectiveness of **game strategies**.
- Assessing policies in **navigation** or **path planning**.

Direct utility estimation

- The agent directly estimates the **utility (value)** of each state based on **observed rewards**.
- It uses **experience** to calculate how good each state is.

Example:

If reaching the goal gives +10 and hitting a wall gives -1, the agent averages rewards from many runs to estimate the utility of each cell in the maze.

Adaptive dynamic programming

- The agent builds a **model of the environment** (transition probabilities and rewards) and then **computes utilities** using that model.
- It adapts its model as it gathers new experience.

Example:

A robot learns how its movements affect its position and updates its internal map for better planning.

Temporal difference learning

- TD learning **combines ideas from direct estimation and ADP**.
- The agent updates the utility of a state based on **differences between successive predictions** rather than waiting until the final outcome.

Formula:

$$U(s) \leftarrow U(s) + \alpha[R(s) + \gamma U(s') - U(s)]$$

R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.

Where:

- U(s)U(s)U(s): current utility of state
- α\alphaα: learning rate
- R(s)R(s)R(s): reward received
- γ\gammaγ: discount factor
- U(s′)U(s')U(s′): utility of the next state

Example:

If a game agent moves one step and gets a small reward, it immediately updates its knowledge of how good that state is — without waiting for the game to end.

Active reinforcement learning

- The agent **actively explores** and **tries different actions** to find the best policy.
- It balances **exploration** (trying new actions) and **exploitation** (using the best-known actions).

Example:

A robot in a maze sometimes takes unknown paths to discover if they might lead to better rewards.

Q learning

- **Q-learning** is one of the most popular active RL algorithms.
- It learns the **quality (Q-value)** of a particular action in a specific state.

Q-value formula:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[R + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

- Q(s,a): current value of taking action *a* in state *s*
- RRR: reward received
- s′s's′: next state
- maxa′Q(s′,a′)\max_{a'} Q(s', a')maxa′Q(s′,a′): best future Q-value
- α\alphaα: learning rate
- γ\gammaγ: discount factor

Goal:

To find the **optimal policy π*** such that it always chooses the action with the **highest Q-value**.

Example of Q-Learning

Suppose a robot vacuum cleaner:

- **States:** Room positions
- **Actions:** Move Left, Right, or Clean

- **Rewards:** +10 for cleaning dirt, -5 for hitting an obstacle

Over time, using Q-learning, it learns which actions in which rooms give the **best long-term rewards** — leading to smarter cleaning.

UNIT V – Reinforcement Learning

Two-Mark Questions

1. What is reinforcement learning (RL)?
2. Define agent and environment in RL.
3. What is passive reinforcement learning?
4. Define direct utility estimation.
5. What is adaptive dynamic programming?
6. Define temporal difference learning.
7. What is active reinforcement learning?
8. Define Q-learning.
9. What is a reward signal in RL?
10. Mention two applications of reinforcement learning.

Five-Mark Questions

1. Explain **passive reinforcement learning** with an example.
2. Describe **direct utility estimation** and its role in RL.
3. Explain **temporal difference learning** in detail.
4. Discuss **active reinforcement learning** and how it differs from passive RL.
5. Explain **Q-learning** algorithm with suitable steps.

Ten-Mark Questions

1. Discuss in detail the **types of reinforcement learning**: passive and active.
2. Explain the **Q-learning algorithm** with its working, equations, and applications.

Compare and explain **Direct Utility Estimation, Adaptive Dynamic Programming, and Temporal Difference Learning**.

R.Subraja., M.Sc.,M.Phil.,
Department of Computer Science,
Alagappa University.